腾讯高性能计算岗位面试准备

GPU 架构・Triton ・TP/SP Attention ・分布式通信・项目深挖

面试备战资料

2026-10-21

目录

第一章 腾讯高性能计算岗位面试准备 **5**
1.1 目标 5
1.2 文件导航 5
1.3 建议的学习方式 5
1.4 三天冲刺版 6
1.5 七天完整版 6
1.6 面试现场的通用解题顺序 7
1.7 完成打勾 7

第二章 项目深挖准备 **8**
2.1 一、每个重点项目填写一张卡片 8
2.2 二、三种时长的回答模板 10
2.3 三、面试官常见追问树 11
2.4 四、项目表达的危险信号 12
2.5 五、项目自测 12

第三章 GPU 架构与性能分析 **14**
3.1 一、先建立整体图景 14
3.2 二、Memory hierarchy 14
3.3 三、Occupancy 与 latency hiding 15
3.4 四、Tensor Core 16
3.5 五、Roofline Model 16
3.6 六、如何解释 FlashAttention 16
3.7 七、Profiler 工作流 17
3.8 八、高频问答 18
3.9 九、闭卷自测 19

第四章 Triton 实战准备 **20**
4.1 一、需要形成的能力 20
4.2 二、Triton 编程模型 20
4.3 三、必会练习一：Vector Add 20
4.4 四、必会练习二：Fused Softmax 21
4.5 五、必会练习三：Matmul 22
4.6 六、必会练习四：RMSNorm 或 LayerNorm 23
4.7 七、Attention/FlashAttention 需要能讲到的程度 23

4.8 八、Autotune . . . 24
4.9 九、正确的 Benchmark . . . 24
4.10 十、调优顺序 . . . 24
4.11 十一、高频追问 . . . 25
4.12 十二、建议的实战产出 . . . 25
4.13 官方资料 . . . 25

第五章 TP/SP Attention：张量与通信推导 26
5.1 一、先声明假设 . . . 26
5.2 二、单卡标准 MHA . . . 26
5.3 三、Tensor Parallel MHA . . . 27
5.4 四、TP 总表 . . . 28
5.5 五、反向传播通信 . . . 29
5.6 六、数值例子 . . . 29
5.7 七、加入经典 Megatron Sequence Parallel . . . 30
5.8 八、不要混淆的两类 Sequence/Context Parallel . . . 31
5.9 九、GQA/MQA 追问 . . . 32
5.10 十、推理与 KV Cache . . . 32
5.11 十一、复杂度检查 . . . 33
5.12 十二、现场推导话术 . . . 33
5.13 十三、常见错误 . . . 33
5.14 十四、白板练习 . . . 33

第六章 通信与分布式高性能计算 35
6.1 一、基本性能模型 . . . 35
6.2 二、四种核心 Collective . . . 35
6.3 三、Ring 通信量 . . . 36
6.4 四、硬件互联层级 . . . 37
6.5 五、通信与计算重叠 . . . 37
6.6 六、并行策略对比 . . . 38
6.7 七、扩展效率 . . . 38
6.8 八、性能指标 . . . 39
6.9 九、典型面试题 . . . 39
6.10 十、现场计算练习 . . . 40
6.11 官方资料 . . . 40

第七章 Vibe Coding 回答与技巧 41
7.1 一、推荐的面试回答 . . . 41
7.2 二、适合使用的场景 . . . 41
7.3 三、不应直接托管给模型的场景 . . . 42
7.4 四、可靠工作流 . . . 42
7.5 五、GPU/Triton 专用检查表 . . . 43
7.6 六、好的提示词结构 . . . 44
7.7 七、常见失败模式 . . . 44
7.8 八、加分表达 . . . 45

第八章 模拟面试题与评分表 **46**
8.1 一、45 分钟模拟流程 46
8.2 二、项目题 46
8.3 三、GPU 架构题 47
8.4 四、TP/SP 白板题 47
8.5 五、Triton 题 48
8.6 六、分布式通信题 48
8.7 七、Vibe Coding 题 48
8.8 八、评分表 49
8.9 九、复盘模板 49
8.10 十、适合向面试官反问 50

第九章 面试前一页速查 **51**
9.1 项目回答 51
9.2 GPU 51
9.3 TP Attention 51
9.4 Megatron SP 52
9.5 Collective 52
9.6 Triton 53
9.7 Vibe Coding 53
9.8 现场推导规则 53
9.9 最后提醒 54

第一章　腾讯高性能计算岗位面试准备

1.1　目标

这套材料针对本轮面试已经暴露出的考察方向：

1. 能把自己的项目讲深，而不只是复述简历。
2. 能从 GPU 架构解释性能现象。
3. 能写、读、调优 Triton kernel。
4. 能现场推导 TP/SP 下 Attention 的张量形状和通信。
5. 能讨论分布式训练中的通信、拓扑和扩展效率。
6. 能理性说明 Vibe Coding 的使用方法与边界。

1.2　文件导航

文件	用途	优先级
01_ 项目深挖准备.md	整理每个核心项目，准备连续追问	P0
02_GPU 架构与性能分析.md	GPU 执行与存储体系、Roofline、Profiler	P0
03_Triton 实战准备.md	Triton 编程模型、Kernel 调优、实战题	P1
04_TP_SP_Attention 推导.md	Attention 在 TP/SP 下的 shape 与通信	P0
05_ 通信与分布式高性能计算.md	Collective、拓扑、重叠、扩展效率	P1
06_Vibe_Coding 回答与技巧.md	开放题回答、工作流和提示词模板	P2
07_ 模拟面试题与评分表.md	模拟题、白板练习、复盘标准	P0
08_ 面试前一页速查.md	面试当天快速复习	P0

P0 的完成标准不是 “看过”，而是能够不看材料在白板上讲出来。

1.3　建议的学习方式

每个知识点按以下闭环完成：

1. **闭卷讲解**：先口述，不会的地方做好标记。
2. **白板推导**：所有张量维度必须自己写一遍。

3. **最小实验**：Triton 或 PyTorch 写一个可验证的小例子。
4. **量化回答**：尽量给公式、shape、数据规模和性能数字。
5. **反向追问**：问自己 “为什么” “代价是什么” “换个硬件还成立吗”。

1.4 三天冲刺版

1.4.1 第一天：项目 + GPU

- 为最重要的两个项目填写项目卡片。
- 各录一次 60 秒版和 5 分钟版项目介绍。
- 复习 GPU 执行层级、内存层级、coalescing、bank conflict、occupancy。
- 用 Roofline 方法分析一个自己项目里的 kernel。

1.4.2 第二天：TP/SP + 通信

- 闭卷推导标准 MHA、TP MHA、TP+SP MHA。
- 用数值例子计算每卡 tensor shape。
- 画出 All-Reduce、All-Gather、Reduce-Scatter、All-to-All 的位置。
- 补充 GQA、KV cache、Ulysses 和 Ring Attention 的区别。

1.4.3 第三天：Triton + 模拟

- 手写 vector add、softmax 的核心代码。
- 能解释 matmul 的 tiling、num_warps、num_stages。
- 完成一轮 45 分钟模拟面试。
- 根据评分表补齐最弱的两项。

1.5 七天完整版

天数	主题	当天必须产出
Day 1	项目一	一页项目卡、60 秒录音、5 分钟录音
Day 2	项目二 + 性能方法	一页项目卡、瓶颈证据表
Day 3	GPU 架构	手画架构图、回答 15 个高频题
Day 4	Triton	两个可运行 kernel、正确性和 benchmark
Day 5	TP/SP Attention	三遍闭卷推导、一次数值计算
Day 6	通信与并行	Collective 对比表、拓扑分析
Day 7	综合模拟	两轮模拟、错题清单、一页速查复习

1.6 面试现场的通用解题顺序

遇到张量或并行推导题时，不要直接猜：

1. 先定义符号与 layout。
2. 说明使用 MHA、GQA 还是 MQA。
3. 说明权重按输入维还是输出维切。
4. 逐算子写输入、权重和输出 shape。
5. 判断结果是完整值还是 partial sum。
6. 只有在需要拼接、求和或换切分方式时加入 collective。
7. 最后补充通信量、显存与反向传播。

1.7 完成打勾

- □ 两个重点项目都有量化数据和 profiler 证据
- □ 能解释自己的优化为什么有效
- □ 能闭卷画 GPU memory hierarchy
- □ 能闭卷推导 TP Attention
- □ 能区分 Megatron SP、Ulysses、Ring/Context Parallel
- □ 能手写一个 Triton reduction kernel
- □ 能解释四种主要 collective
- □ 有一段成熟的 Vibe Coding 回答
- □ 完成至少两次计时模拟

第二章　项目深挖准备

2.1　一、每个重点项目填写一张卡片

不要直接背下面的标题。先把事实和数字填完整，再组织语言。

2.1.1　1. 一句话定义

> 我在【业务/研究背景】下，为了解决【具体瓶颈】，负责了【核心模块】，最终使【关键指标】从【基线】提升到【结果】。

2.1.2　2. 项目背景

- 使用场景：
- 用户或下游系统：
- 模型/算法：
- 硬件：GPU 型号、GPU 数、单机/多机、互联方式
- 软件栈：PyTorch/CUDA/Triton/NCCL/框架版本
- 典型输入：batch、sequence length、hidden size、dtype
- 优化前的核心问题：

2.1.3　3. 个人职责

- 我独立完成的部分：
- 我主导设计、别人协作实现的部分：
- 我修改的核心模块：
- 我做出的关键技术决策：
- 我没有负责、但能说明接口的部分：

面试时主动划清边界。比含糊地把团队成果都归给自己更可信。

2.1.4　4. Baseline 与瓶颈证据

项目	内容
Baseline	原实现、官方实现或竞品
性能指标	latency、throughput、tokens/s、TFLOPS、MFU
正确性指标	最大误差、平均误差、任务指标
Profiler 工具	Nsight Systems、Nsight Compute、PyTorch Profiler
GPU 利用率	SM、Tensor Core、memory bandwidth
通信占比	NCCL 时间、等待时间、是否与计算重叠
显存构成	参数、梯度、优化器、activation、临时 buffer
主要瓶颈	compute / memory / launch / synchronization / communication

2.1.5 5. 方案选择

至少准备三种方案：

方案	优点	缺点	为什么选或不选
方案 A：最终方案			
方案 B：替代方案			
方案 C：保守方案			

不要只说 “效果最好”。要从改动风险、开发成本、硬件适配、数值精度和可维护性解释。

2.1.6 6. 优化前后对比

指标	优化前	优化后	提升	测试条件
P50 latency				
P99 latency				
Throughput				
Peak memory				
MFU				
多卡扩展效率				

所有性能数字都必须能回答：

- 在什么 shape 和硬件上测的？
- warmup 多少次，正式测多少次？
- 同步点放在哪里？
- 对比是否使用相同精度和相同算法？
- 提升来自算法变化，还是单纯减少了工作量？

2.1.7 7. 正确性验证

- 参考实现是什么?
- 使用 fp32、fp16、bf16 还是 fp8?
- 误差阈值如何选择?
- 是否覆盖极小值、极大值、非整齐 shape、空 batch 等边界?
- 并行后如何确认 rank 间结果一致?
- 是否做端到端回归，而不只是 kernel 单测?

2.1.8 8. 失败与复盘

- 最初的错误判断:
- 一个没有收益的优化:
- 一次数值或并发问题:
- 如何定位:
- 最后学到了什么:
- 如果重新做，会先测什么:

2.2 二、三种时长的回答模板

2.2.1 60 秒版本

1. 10 秒：问题和规模。
2. 15 秒：自己负责什么。
3. 20 秒：核心瓶颈与方案。
4. 10 秒：量化结果。
5. 5 秒：最难点或经验。

示例骨架:

> 这个项目是在【硬件/规模】上优化【任务】。原实现的主要问题是【瓶颈】，我通过【工具和指标】确认它占总耗时【比例】。我主要负责【模块】，采用【方案】，核心是【技术原理】。在相同【shape、精度和硬件】下，吞吐从【A】提升到【B】，显存下降【C】。最难的地方是【问题】，最后通过【验证方法】解决。

2.2.2 5 分钟版本

按 “背景 → 基线 → 证据 → 方案 → 结果 → 反思” 展开。每个技术决策都要能接一句:

> 这样选择的原因是……；它的代价是……；我们通过……验证。

2.2.3 15 分钟深挖版本

准备回答：

- 系统数据流和关键模块接口
- 性能模型与理论上限
- 关键 kernel 的 shape、访存和计算量
- 多卡的切分策略与 collective
- 异常处理、容错和工程约束
- 替代设计、失败实验与未来优化

2.3 三、面试官常见追问树

2.3.1 “为什么快？”

继续追问可能是：

- 少算了什么？
- 少读写了多少 HBM？
- 减少了多少 kernel launch？
- Tensor Core 利用率为何提高？
- 是不是把时间转移到了别处？
- 不同 shape 下都成立吗？

回答时把收益归因到可验证机制：

- FLOPs 减少
- HBM bytes 减少
- fusion 减少中间张量与 launch
- layout 改善 coalescing
- tiling 提高数据复用
- pipeline 隐藏 memory/communication latency

2.3.2 “怎么确定瓶颈？”

推荐回答顺序：

1. Nsight Systems 看端到端时间线和空洞。
2. 找出耗时最大的 kernel、通信与同步。
3. Nsight Compute 看算力、带宽、occupancy、stall reason。
4. 结合 Roofline 判断 compute-bound 或 memory-bound。
5. 做控制变量实验验证，而不是只依赖单一指标。

2.3.3 “为什么不用另一个方案？”

从以下维度比较：

- 性能收益
- 显存开销
- 通信开销
- 数值精度
- shape 泛化
- 硬件兼容
- 开发和维护成本

2.3.4 “你个人做了什么？”

使用动词和证据：

- 我定位……
- 我设计……
- 我实现……
- 我用……验证……
- 我推动团队在……之间做出选择……

避免只说 “我们做了”。

2.4 四、项目表达的危险信号

- 只有百分比，没有 baseline 和测试条件。
- 说 “GPU 利用率很低”，但不知道指 SM、Tensor Core 还是时间占比。
- 说 “通信很慢”，但不知道 tensor 大小、collective 和拓扑。
- 说 “做了算子融合”，但解释不了减少了哪些读写与 launch。
- 只会讲成功版本，不知道替代方案和失败过程。
- 混淆 wall-clock、kernel time、GPU time 和 CPU launch time。
- 把团队成果全部说成个人成果。

2.5 五、项目自测

对每个项目闭卷回答：

1. 一句话说明价值。
2. 画出模块图和数据流。
3. 写出最关键的三个 tensor shape。
4. 给出一个性能模型。
5. 给出 profiler 证据。

6. 解释优化的因果链。
7. 给出两个具体数字。
8. 说明一个失败方案。
9. 说明精度如何验证。
10. 说明扩到更多 GPU 会先遇到什么问题。

第三章　GPU 架构与性能分析

3.1　一、先建立整体图景

面试时建议从下面这条层级链解释：

> GPU → GPC → SM → warp scheduler / execution units → CUDA Core / Tensor Core / LD-ST units

编程层级：

> grid → block/CTA → warp → thread

- 一个 warp 通常包含 32 个线程。
- block 被调度到某个 SM 后，其 warp 在该 SM 上执行。
- SIMT 表示线程执行同一指令流，但每个线程有自己的寄存器和数据。
- 分支导致同一 warp 内线程走不同路径时，会出现 divergence，路径通常需要分批执行。

不要死背某一代 GPU 的 SM 数量和单元数量。面试前根据目标硬件单独核对 A100/H100/其他型号的官方规格。

3.2　二、Memory hierarchy

层级	范围	特征	常见优化
Register	每线程	最快，容量有限	控制 live range 和 registe
Shared Memory	每 block	程序显式管理，可跨线程复用	tiling、避免 bank conflic
L1 Cache	每 SM	低延迟，和 shared memory 关系依架构而异	提高局部性
L2 Cache	全 GPU	所有 SM 共享	数据复用、访问顺序
HBM/Global Memory	全 GPU	容量大、延迟高、带宽高	coalescing、减少读写
Host/Peer Memory	CPU 或其他 GPU	受 PCIe/NVLink 和拓扑影响	批量传输、异步重叠

3.2.1　Coalescing

同一个 warp 的线程访问连续且对齐的地址时，硬件可以用较少的 memory transaction 完成访问。

典型坏例子：

- stride 很大
- AoS 导致访问字段分散
- 非对齐访问
- transpose 时直接按不连续方向读写

改进方法：

- 改 layout
- 向量化 load/store
- 用 shared memory 做 tile 和转置
- 让连续线程对应连续元素

3.2.2 Shared-memory bank conflict

shared memory 被划分为多个 bank。一个 warp 的多个线程同时访问同一 bank 的不同地址时，访问可能串行化。

常见解决：

- 给二维 tile 增加 padding，例如宽度从 N 改为 N+1。
- 改变 index 映射。
- 利用硬件支持的 broadcast 场景。

3.3 三、Occupancy 与 latency hiding

Occupancy 通常表示：

> 当前 SM 上活跃 warp 数 / 架构允许的最大活跃 warp 数

限制来源：

- 每线程寄存器太多
- 每 block shared memory 太多
- block 太大
- 架构限制的 block/warp 数

高 occupancy 不等于高性能：

- kernel 可能已经 compute-bound。
- 指令依赖链过长，增加 warp 也不一定解决。
- 内存访问不合并或 cache miss 严重。
- 更多并发可能导致 cache 抖动。

正确表达是：

> Occupancy 是 latency hiding 的手段之一，不是最终目标。应该结合吞吐、stall reason、寄存器使用和带宽判断。

3.4 四、Tensor Core

Tensor Core 面向小矩阵的 multiply-accumulate。大型 GEMM 会被切成 block tile、warp tile 和 instruction tile。

需要掌握：

- 输入 dtype、累加 dtype 与输出 dtype 可能不同。
- 矩阵维度和 layout 会影响是否能高效使用 Tensor Core。
- padding 虽然增加少量 FLOPs，但有时能换来更好的指令效率。
- fp16/bf16/fp8 的吞吐、动态范围和精度权衡。
- accumulation 通常需要更高精度以控制误差。

如果被问 Ampere 与 Hopper：

- Ampere 重点：Tensor Core、异步拷贝、较成熟的 TF32/BF16 支持。
- Hopper 重点：更强 Tensor Core、Transformer Engine、TMA、异步事务和 warp specialization。
- 具体峰值和单元数量应以目标型号官方规格为准。

3.5 五、Roofline Model

定义：

> Arithmetic Intensity = FLOPs / 从主要存储层搬运的 Bytes

性能上限：

> Performance ⩽ min(Peak Compute, Arithmetic Intensity × Memory Bandwidth)

判断方法：

- 算术强度低，性能随带宽上升：倾向 memory-bound。
- 算术强度高，接近计算峰值：倾向 compute-bound。
- 实际远低于两条上限：检查 launch、依赖、分支、occupancy、layout、同步等问题。

注意 “Bytes” 取决于分析的是 HBM、L2 还是 shared memory roofline，必须说明层级。

3.6 六、如何解释 FlashAttention

标准 Attention：

1. 计算 score = QK^T。
2. 将 S×S score 写入 HBM。
3. 读回 score 做 softmax，再写回。
4. 再读 softmax 结果与 V 相乘。

FlashAttention：

- 把 Q/K/V 分块搬到 SRAM/shared memory。
- 使用 online softmax，维护每行的最大值和归一化因子。
- 不需要将完整 S×S score 矩阵落到 HBM。
- 精确计算标准 Attention，不是近似 Attention。

主要收益来自减少 HBM I/O，而不是降低主项 FLOPs。

Online softmax 合并两个 block 时：

$$m_{new} = \max(m_{old}, m_{block})$$

$$l_{new} = \exp(m_{old}-m_{new}) \times l_{old} + \exp(m_{block}-m_{new}) \times l_{block}$$

累积输出也要按新的最大值重新缩放。

3.7 七、Profiler 工作流

3.7.1 第一步：端到端

使用 Nsight Systems 或 PyTorch Profiler：

- CPU 是否及时提交 kernel
- 是否有大量小 kernel
- GPU 时间线是否有空洞
- H2D/D2H 是否阻塞
- NCCL 是否和计算重叠
- 是否存在意外同步

3.7.2 第二步：单 kernel

使用 Nsight Compute：

- achieved FLOPs / Tensor Core 使用
- DRAM 和 L2 吞吐
- occupancy
- registers per thread
- shared memory per block
- warp stall reasons
- global load/store efficiency
- 指令和访存 roofline

3.7.3 第三步：控制变量实验

- 改 shape，观察复杂度变化。
- 改 dtype，观察 Tensor Core 与带宽变化。

- 固定 FLOPs 改 layout。
- 固定单卡 workload 改 GPU 数。
- 暂时关闭某项 fusion 或 overlap，验证收益归因。

3.8 八、高频问答

3.8.1 为什么 GPU 适合深度学习?

大量规则且可并行的矩阵与向量运算，具有较高数据并行度；GPU 提供大量吞吐型执行单元、高 HBM 带宽和专用 Tensor Core。

3.8.2 Warp divergence 一定很慢吗?

不一定。影响取决于分支比例、分支长度和是否发生在同一 warp。若分支在 warp 间一致，代价很小；短分支也可能被编译器谓词化。

3.8.3 Shared memory 一定比 L1 快吗?

不要绝对化。shared memory 的主要优势是显式控制和可预测复用；实际延迟与架构、冲突和访问方式有关。

3.8.4 为什么 kernel fusion 能加速?

减少中间 tensor 的 HBM 读写、kernel launch 和同步；代价可能是寄存器压力、代码复杂度和通用性下降。

3.8.5 为什么小矩阵跑不满 GPU?

并行任务数不足、launch 开销占比高、tile 利用率低，或维度不适合 Tensor Core。

3.8.6 如何区分 memory-bound 与 compute-bound?

先做 Roofline 估计，再结合 profiler 的 FLOPs、带宽和 stall 指标，并通过改变计算量或访存量做控制实验。

3.9 九、闭卷自测

1. 画出 memory hierarchy，并说明作用域和生命周期。
2. 举例解释 coalescing 和 bank conflict。
3. 解释 register pressure 如何影响 occupancy。
4. 解释 occupancy 为什么不是越高越好。
5. 用 Roofline 判断一个 elementwise kernel。
6. 解释 matmul 为什么有较高 arithmetic intensity。
7. 解释 FlashAttention 的 online softmax。
8. 说明如何用 Nsight Systems 和 Nsight Compute 定位瓶颈。
9. 对比 CUDA Core 和 Tensor Core。
10. 解释 fusion 可能变慢的原因。

第四章　Triton 实战准备

4.1　一、需要形成的能力

面试目标不是背 API，而是能完成下面的推理：

1. 把一个算子映射为多个 Triton program instance。
2. 决定每个 program 负责的数据 tile。
3. 设计连续、对齐、可合并的 load/store。
4. 处理边界、reduction 和数值稳定性。
5. 根据寄存器、shared memory、occupancy 调整参数。
6. 用正确性测试和 benchmark 证明结果。

4.2　二、Triton 编程模型

- tl.program_id(axis) 获取当前 program instance 的编号。
- 一个 program 通常处理一个 block/tile，而不是一个 CUDA thread。
- tl.arange 生成 block 内的向量化 offset。
- tl.load/tl.store 进行向量化访存。
- mask 处理非整除边界。
- tl.max、tl.sum 等完成 program 内 reduction。
- tl.dot 使用适合硬件的矩阵乘累加路径。
- num_warps 控制一个 program 使用的 warp 数。
- num_stages 常用于控制软件流水和异步加载深度。

回答 Triton 与 CUDA 的差异时：

> CUDA 更显式地控制 thread/warp/shared memory；Triton 以 block 级向量程序描述计算，由编译器完成一部分线程映射、访存合并和低层调度。Triton 提高开发效率，但 tile、layout、访存和资源占用仍需要开发者理解。

4.3　三、必会练习一：Vector Add

需要能手写：

```
import torch
import triton
import triton.language as tl


@triton.jit
def add_kernel(x_ptr, y_ptr, out_ptr, n_elements: tl.constexpr,
               BLOCK_SIZE: tl.constexpr):
    pid = tl.program_id(axis=0)
    offsets = pid * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE)
    mask = offsets < n_elements

    x = tl.load(x_ptr + offsets, mask=mask)
    y = tl.load(y_ptr + offsets, mask=mask)
    tl.store(out_ptr + offsets, x + y, mask=mask)


def add(x, y):
    out = torch.empty_like(x)
    n = out.numel()
    grid = (triton.cdiv(n, 256),)
    add_kernel[grid](x, y, out, n, BLOCK_SIZE=256)
    return out
```

必须能解释：

- 为什么 grid 使用向上取整。
- 为什么最后一个 block 需要 mask。
- 连续线程/lanes 是否访问连续地址。
- BLOCK_SIZE 太小或太大会有什么影响。

4.4 四、必会练习二：Fused Softmax

每个 program 处理一行的基础版本：

```
@triton.jit
def softmax_kernel(x_ptr, y_ptr, n_cols: tl.constexpr,
                   BLOCK_SIZE: tl.constexpr):
    row = tl.program_id(axis=0)
    offsets = tl.arange(0, BLOCK_SIZE)
    mask = offsets < n_cols

    x = tl.load(
        x_ptr + row * n_cols + offsets,
```

```
        mask=mask,
        other=-float("inf"),
    )
    x = x - tl.max(x, axis=0)
    numerator = tl.exp(x)
    denominator = tl.sum(numerator, axis=0)
    y = numerator / denominator

    tl.store(y_ptr + row * n_cols + offsets, y, mask=mask)
```

需要回答：

- 为什么减去最大值：避免 exp 溢出，提高数值稳定性。
- 为什么 padding 区域填负无穷：它对 max 和 exp-sum 不产生有效贡献。
- 为什么 BLOCK_SIZE 常取不小于 n_cols 的 2 的幂：便于 reduction 映射。
- 行很长、超过单 program 合适范围怎么办：分块 reduction、多阶段 kernel 或换算法。
- 为什么比多个 PyTorch op 快：减少 launch 和中间 tensor 的 HBM 往返。

4.5 五、必会练习三：Matmul

假设：

$$C[M,N] = A[M,K] \times B[K,N]$$

每个 program 负责一个 BLOCK_M × BLOCK_N 的 C tile，沿 K 维循环 BLOCK_K：

1. 计算当前 M/N tile 的 offsets。
2. 加载 A 的 BLOCK_M × BLOCK_K。
3. 加载 B 的 BLOCK_K × BLOCK_N。
4. 使用 tl.dot 累加到 fp32 accumulator。
5. K 循环结束后转换到目标 dtype 并写回。

参数作用：

参数	增大后的潜在收益	风险
BLOCK_M/BLOCK_N	数据复用增加、program 数减少	尾块浪费、寄存器增加
BLOCK_K	单次计算更多	pipeline 和资源压力增加
num_warps	并发和协作能力提高	每 program 资源占用提高
num_stages	更好隐藏加载延迟	shared memory 使用增加

必须知道 grouped ordering 的目的：改变 program 的 tile 遍历顺序，提高 A/B tile 在 cache 中的复用。

4.6 六、必会练习四：RMSNorm 或 LayerNorm

RMSNorm：

$$r = \mathrm{rsqrt}(\mathrm{mean}(x^2) + \mathrm{eps})$$

$$y = x \times r \times \mathrm{weight}$$

面试点：

- reduction 使用 fp32 累加。
- 一行是否能由一个 program 处理。
- hidden size 很大时如何分块。
- 是否融合 residual、bias、dropout 或量化。
- forward 容易，backward 中需要哪些 reduction。

4.7 七、Attention/FlashAttention 需要能讲到的程度

- Q、K、V 如何分 block。
- 为什么不能直接存完整 score。
- online softmax 如何跨 K/V block 合并。
- causal mask 和边界 mask 的区别。
- 每个 block 的 Q 在循环中保持，K/V 分块流过。
- backward 为什么更复杂，可能需要重算部分中间量。
- head dimension、sequence length、causal/non-causal 会影响最佳配置。

不要求现场从零写完整 production FlashAttention，但要能写出伪代码和状态变量：

```
for each Q block:
    m = -inf
    l = 0
    acc = 0
    for each K/V block:
        score = Q @ K.T * scale
        apply mask
        m_new = max(m, rowmax(score))
        p = exp(score - m_new)
        alpha = exp(m - m_new)
        l = alpha * l + rowsum(p)
        acc = alpha * acc + p @ V
        m = m_new
    output = acc / l
```

4.8 八、Autotune

候选配置要覆盖合理的 tile 和资源组合，而不是盲目枚举：

- BLOCK_M/BLOCK_N/BLOCK_K
- num_warps
- num_stages

Autotune key 应选择真正改变最佳配置的维度，如 M/N/K 或 hidden size 的分段。注意：

- 首次编译和 autotune 时间不能计入稳定态 benchmark。
- 动态 shape 太多时，配置爆炸会增加缓存和启动成本。
- 某个 shape 最快的配置不一定对所有 shape 都好。

4.9 九、正确的 Benchmark

检查清单：

- warmup 后再测。
- GPU 操作是异步的，计时边界要正确同步或使用 CUDA Event。
- 报告 median，并可补充 P20/P80 或 P50/P95。
- baseline 使用相同 dtype、layout、算法语义。
- 分别测不同 shape，避免只挑有利样本。
- 同时报告 latency、GB/s 或 TFLOPS。
- 小 kernel 要考虑 launch 开销。
- 验证输出再测性能。

带宽估算示例，vector add 至少读 x、读 y、写 out：

$$\text{Bytes} \approx 3 \times \text{n_elements} \times \text{element_size}$$

有效带宽：

$$\text{Effective Bandwidth} = \text{Bytes} / \text{elapsed_time}$$

4.10 十、调优顺序

1. 先确保正确性和稳定性。
2. 明确理论 FLOPs 和 Bytes。
3. 确定 compute-bound 或 memory-bound。
4. 检查 load/store 是否连续且对齐。
5. 设计 tile 和数据复用。
6. 调整 num_warps、num_stages。
7. 检查 register spilling 和 occupancy。
8. 对多个真实 shape 做 autotune。

9. 用 profiler 验证收益机制。

4.11 十一、高频追问

1. Triton 为什么能自动实现 coalescing?
2. 一个 Triton program 和 CUDA block 有什么关系?
3. tl.arange 的结果物理上放在哪里?
4. 为什么 reduction 容易增加 register pressure?
5. mask 会不会带来分支开销?
6. 为什么融合过多反而变慢?
7. num_warps 和 num_stages 分别控制什么?
8. 如何定位 Triton kernel 的性能回退?
9. 如何处理非连续 tensor?
10. 如何证明 kernel 在不同 shape 和 dtype 下都正确?

4.12 十二、建议的实战产出

- □ vector_add.py：正确性 + 带宽
- □ softmax.py：与 torch.softmax 对比
- □ matmul.py：多个 M/N/K shape 的 TFLOPS
- □ rmsnorm.py：融合版本与 PyTorch baseline
- □ benchmark.csv：硬件、版本、shape、dtype、配置、结果
- □ profiler 截图：至少解释一次优化前后变化

4.13 官方资料

- Triton Tutorials：https://triton-lang.org/main/getting-started/tutorials/
- Triton Language API：https://triton-lang.org/main/python-api/triton.language.html
- NVIDIA Nsight Compute：https://docs.nvidia.com/nsight-compute/

第五章　TP/SP Attention：张量与通信推导

这是本轮面试最值得优先练熟的部分。目标是能在白板上边解释边推导，而不是背一张结果表。

5.1　一、先声明假设

建议开场先说：

> 我先假设标准 Multi-Head Self-Attention，Q/K/V head 数相同；权重用 [in_dim, out_dim] 表示；输入采用 [B,S,H]；TP 按 attention heads 切分，并且 H 和 head 数都能被 TP degree 整除。

符号：

符号	含义
B	batch size
S	sequence length
H	hidden size
A	attention head 数
D	head dimension，D=H/A
T	tensor parallel degree
P	独立 sequence/context parallel degree

如果面试官使用 [S,B,H]，只需要交换前两维，推导逻辑不变。

5.2　二、单卡标准 MHA

输入：

> X: [B,S,H]

QKV projection：

> W_qkv: [H,3H]
>
> QKV = X × W_qkv: [B,S,3H]

拆分并 reshape：

Q,K,V: [B,A,S,D]

Attention：

Score = Q × K^T: [B,A,S,S]

Prob = softmax(Score): [B,A,S,S]

Context = Prob × V: [B,A,S,D]

合并 heads：

Context: [B,S,H]

输出投影：

W_o: [H,H]

Y: [B,S,H]

5.3 三、Tensor Parallel MHA

TP 沿 QKV 的输出维切分，也就是每张卡负责 A/T 个 heads。

5.3.1 1. QKV Column Parallel

输入 X 在 TP group 内复制：

X: [B,S,H]

每卡权重：

W_qkv_rank: [H,3H/T]

每卡输出：

QKV_rank: [B,S,3H/T]

拆分：

Q_rank,K_rank,V_rank: [B,A/T,S,D]

这一层 forward 不需要通信，因为每张卡可以独立算出自己负责的输出列。

5.3.2 2. 本地 Attention

每卡：

Score_rank: [B,A/T,S,S]

Prob_rank: [B,A/T,S,S]

Context_rank: [B,A/T,S,D]

因为一个 head 的 D 维没有被拆开，每张卡持有完整序列上的若干完整 heads，所以 QK^T、softmax 和 PV 都可以本地完成。

合并本地 heads：

Context_rank: [B,S,H/T]

5.3.3 3. Output Row Parallel

输出投影沿输入维切分：

W_o_rank: [H/T,H]

每张卡产生 partial sum：

Y_partial_rank = Context_rank × W_o_rank: [B,S,H]

最终：

Y = AllReduce_sum(Y_partial_rank): [B,S,H]

为什么是求和而不是拼接：

完整矩阵乘法在被切分的输入维上做了分块，每个 rank 只计算 K 维的一部分贡献，因此各卡结果形状相同，数值上需要相加。

5.4 四、TP 总表

阶段	每卡 shape	是否通信
输入 X	[B,S,H]	已复制
W_qkv	[H,3H/T]	无
QKV	[B,S,3H/T]	无
Q/K/V	[B,A/T,S,D]	无
Score/Prob	[B,A/T,S,S]	无
Context	[B,S,H/T]	无
W_o	[H/T,H]	无
Y_partial	[B,S,H]	是 partial sum
Y	[B,S,H]	All-Reduce

一句话总结：

Column-parallel QKV 的 forward 无通信，本地 heads 完成 Attention，row-parallel 输出投影后对 [B,S,H] 做 All-Reduce。

5.5 五、反向传播通信

5.5.1 Output Row Parallel

- forward：partial output 需要 All-Reduce。
- backward：上游 dY 在各 rank 已有完整副本；每卡可得到本地 dContext，一般不需要为 dContext 再通信。

5.5.2 QKV Column Parallel

- forward：输出本来就是分片，无通信。
- backward：每卡从本地 QKV 分片计算一个 dX partial。
- 因为 X 在 forward 是复制的，所以这些 dX partial 需要求和。

因此，普通 TP Attention 中：

- forward 的关键 collective：输出投影后的 All-Reduce。
- backward 的关键 collective：QKV 投影对 dX 的 All-Reduce。

讨论通信时要说明是在 forward、backward 还是两者总计。

5.6 六、数值例子

设：

B=8，S=4096，H=8192，A=64，D=128，T=8

每卡负责：

A/T = 8 个 heads

各 tensor：

Tensor	每卡 shape
X	[8,4096,8192]
W_qkv_rank	[8192,3072]
QKV_rank	[8,4096,3072]
单个 Q/K/V	[8,8,4096,128]
Score	[8,8,4096,4096]
Context	[8,4096,1024]
W_o_rank	[1024,8192]
Y_partial	[8,4096,8192]

Score 每卡有 1,073,741,824 个元素；如果直接以 bf16 保存，仅该 tensor 就约 2 GiB。这也说明了 FlashAttention 避免将完整 S×S score 落到 HBM 的价值。

输出 All-Reduce 的逻辑 tensor 大小：

$N = B \times S \times H$ 个元素

Ring All-Reduce 常用带宽模型中，每卡发送的数据量约为：

$2 \times (T-1)/T \times N \times element_size$

使用该公式时，要明确统计口径是发送字节、接收字节还是链路总流量。

5.7 七、加入经典 Megatron Sequence Parallel

先确认定义：

这里先讨论 Megatron-style sequence parallel，它通常在 TP group 内沿 sequence 切 activation，并不等同于把 Attention 的上下文长度真正分给独立设备计算。

SP 区域的输入：

X_sp_rank: [B,S/T,H]

5.7.1 1. 进入 QKV Column Parallel

QKV 投影需要完整 sequence 上的输入，因此先沿 sequence：

All-Gather: [B,S/T,H] → [B,S,H]

然后本地执行 column-parallel QKV：

QKV_rank: [B,S,3H/T]

Q/K/V_rank: [B,A/T,S,D]

Attention 部分仍然是：

Score_rank: [B,A/T,S,S]

说明经典 Megatron SP 没有进一步把每个 head 的 S×S Attention 矩阵切小。

5.7.2 2. 离开 Output Row Parallel

每卡计算：

Y_partial_rank: [B,S,H]

不再做得到完整复制结果的 All-Reduce，而是沿 sequence 做：

Reduce-Scatter: [B,S,H] → [B,S/T,H]

这样结果直接回到 sequence-sharded 状态，供 residual、dropout、LayerNorm 等操作使用。

5.7.3 3. 总表

阶段	每卡 shape	Collective
SP 输入	[B,S/T,H]	无
QKV 前	[B,S,H]	All-Gather(sequence)
Q/K/V	[B,A/T,S,D]	无
Attention	[B,A/T,S,S]	无
O projection partial	[B,S,H]	partial sum
SP 输出	[B,S/T,H]	Reduce-Scatter(sequence)

反向传播中的 collective 是相应线性算子的伴随方向：

- forward All-Gather 对应 backward Reduce-Scatter。
- forward Reduce-Scatter 对应 backward All-Gather。

5.7.4 4. SP 的价值

- LayerNorm、dropout、residual 等 activation 按 sequence 分片，显存约按 T 缩小。
- 避免在这些区域每卡都保存完整 [B,S,H]。
- All-Gather + Reduce-Scatter 的总带宽量级与一个 Ring All-Reduce 相近，但产生的是后续真正需要的分片布局。
- 它主要解决非 TP 区域的 activation 冗余，不直接消除 S^2 Attention 开销。

5.8 八、不要混淆的两类 Sequence/Context Parallel

5.8.1 Ulysses

设独立 SP degree 为 P。开始时每卡拥有部分序列、全部 heads：

Q/K/V: [B,S/P,A,D]

通过 All-to-All 把布局转成完整序列、部分 heads：

[B,S/P,A,D] → [B,S,A/P,D]

每卡独立计算完整序列上的一部分 heads，完成后用反向 All-to-All 变回 sequence-sharded 布局。

关键通信：两次 All-to-All。

5.8.2 Ring Attention / Context Parallel

每卡保留：

Q/K/V local: [B,A,S/P,D]

Q 保持本地，K/V block 在设备间环形传递；每一步计算局部 score block，并用 online softmax 合并状态。

优点：

- 不要求每卡同时保存完整 K/V 序列。
- 可扩展到很长上下文。

代价：

- P2P 通信与调度更复杂。
- causal mask、负载均衡和通信重叠更难。

5.9 九、GQA/MQA 追问

设 Query heads 为 A_q，KV heads 为 A_kv：

- GQA：A_kv < A_q。
- MQA：A_kv = 1。

如果 A_kv 能被 T 整除，可按 KV heads 切分。

如果 A_kv < T 或不能整除，可能选择：

- 在多个 TP rank 复制 K/V。
- 限制 Attention 使用的 TP degree。
- 使用专门的 head-to-rank 映射。
- 将 TP 与其他并行维度组合。

不要机械写成 A_kv/T；先检查整除关系和实现策略。

5.10 十、推理与 KV Cache

Prefill：

- Q 的长度通常大于 1。
- Attention 计算量较大，形态接近训练 forward。

Decode：

- 当前 Q 常为 [B,A/T,1,D]。
- 本地 KV cache 可写为 [B,A_kv/T,S_cache,D]，前提是 KV heads 可切。
- Score 为 [B,A/T,1,S_cache]。
- Attention 核心一般仍可在本地 heads 上执行。
- TP 通信主要仍出现在输出投影等需要合并 partial sum 的位置。

Decode 常更容易受 KV cache 带宽、batch 调度和通信延迟影响。

5.11 十一、复杂度检查

标准 MHA 的主要 FLOPs，按一次乘加计 2 FLOPs：

- QK^T：约 $2BS^2H$。
- Prob×V：约 $2BS^2H$。
- Attention 核心合计：约 $4BS^2H$。
- QKV projection：约 $6BSH^2$。
- Output projection：约 $2BSH^2$。

Head-parallel TP 下，各卡的这些计算量理想情况下约除以 T。

显式保存 score/prob 的 activation 规模为：

$$O(BAS^2/T)$$

5.12 十二、现场推导话术

我先固定 layout 为 [B,S,H]，假设标准 MHA 且 head 数可以被 TP degree 整除。QKV 是 column parallel，所以每卡权重是 [H,3H/T]，输出是 [B,S,3H/T]，reshape 后每卡有 A/T 个完整 heads。因为 sequence 没切，QK、softmax 和 PV 都能本地完成。Context 合并为 [B,S,H/T]。O projection 是 row parallel，权重为 [H/T,H]，每卡输出 [B,S,H] 的 partial sum，因此需要 All-Reduce。如果加的是 Megatron SP，层间 activation 是 [B,S/T,H]；QKV 前 All-Gather 回完整 S，O projection 后以 Reduce-Scatter 直接得到 [B,S/T,H]。

5.13 十三、常见错误

- 没有说明权重 shape 的方向。
- 把 column-parallel 输出误认为需要 All-Gather。
- 把 row-parallel 输出当成 concat，而不是 partial sum。
- 忘记 Attention score 的两个 S 维。
- 将 Megatron SP 误认为 score 也变成 [S/T,S/T]。
- 不询问 SP 是 Megatron、Ulysses 还是 Ring/Context Parallel。
- GQA 中不检查 KV head 数是否能被 TP degree 整除。
- 只说 forward 通信，不考虑 backward。
- 通信量不说明 dtype 和统计口径。

5.14 十四、白板练习

分别对以下配置推导所有 shape 和通信：

1. B=2，S=2048，H=4096，A=32，T=4。

2. B=8，S=4096，H=8192，A=64，T=8。
3. GQA：A_q=64，A_kv=8，T=8。
4. MQA：A_q=64，A_kv=1，T=8。
5. TP=8 加 Megatron SP。
6. 独立 TP=4、Ulysses SP=8，说明两个 process group 的职责。

每题必须回答：

- 每卡权重 shape
- 每卡 activation shape
- forward collective
- backward collective
- 通信 tensor 大小
- 哪部分计算/显存按并行度缩小

第六章　通信与分布式高性能计算

6.1　一、基本性能模型

常用简化模型：

$$T_{comm} \approx \alpha \times 消息/轮次开销 + \beta \times 传输字节数$$

- α：latency 项，受软件栈、同步和网络跳数影响。
- β：每字节时间，约等于 1 / 有效带宽。

小消息通常更受 latency 影响，大消息更受 bandwidth 影响。

总训练时间不要只写成计算加通信，因为二者可能重叠：

$$T_{step} \approx T_{exposed_compute} + T_{exposed_comm} + T_{unhidden_overhead}$$

真正需要优化的是暴露在关键路径上的通信，而不是所有 NCCL 时间之和。

6.2　二、四种核心 Collective

6.2.1　All-Reduce

每个 rank 有相同 shape 的 tensor，最终每个 rank 都得到逐元素归约后的完整 tensor。

典型用途：

- Data Parallel 梯度同步
- Tensor Parallel row-parallel linear 输出求和

等价结构：

$$\text{All-Reduce} = \text{Reduce-Scatter} + \text{All-Gather}$$

6.2.2　All-Gather

每个 rank 有完整 tensor 的一个分片，最终每个 rank 都得到所有分片拼接后的完整 tensor。

典型用途：

- Megatron SP 进入需要完整 sequence 的 TP 区域
- FSDP 收集参数

6.2.3 Reduce-Scatter

各 rank 的相同 shape tensor 先逐元素归约，再把结果按某个维度分片到各 rank。

典型用途：

- SP 输出
- ZeRO/FSDP 的梯度归约与分片

6.2.4 All-to-All

每个 rank 将不同分片发给不同 rank，并接收来自所有 rank 的相应分片。

典型用途：

- MoE token dispatch
- Ulysses sequence-head layout 转换

它比简单 ring collective 更依赖网络拓扑、并发连接和流量均衡。

6.3 三、Ring 通信量

设全局逻辑 tensor 大小为 N bytes，参与 rank 数为 p。

Ring Reduce-Scatter：

每卡发送约 $(p-1)/p \times N$ bytes

Ring All-Gather：

每卡发送约 $(p-1)/p \times N$ bytes

Ring All-Reduce：

每卡发送约 $2(p-1)/p \times N$ bytes

其优点是对大消息的带宽利用较好，缺点是需要 O(p) 个步骤，rank 多、小消息时 latency 项较明显。

树形算法步骤约为 O(log p)，更适合降低小消息 latency；具体选择由消息大小、拓扑和 NCCL 算法决定。

说明通信量时必须交代：

- N 是每卡本地输入大小还是全局总大小。
- 统计发送、接收还是二者之和。
- dtype 对应的 element size。
- 是否把跨节点和节点内链路分开计算。

6.4 四、硬件互联层级

常见路径：

- 同一 GPU 内：SM 通过 L2/HBM 共享数据。
- 同机 GPU 间：NVLink/NVSwitch 或 PCIe。
- 跨机：NIC + InfiniBand/RoCE。
- GPU Direct RDMA：NIC 与 GPU memory 直接传输，减少 CPU staging。

拓扑会影响：

- 实际链路带宽
- 是否经过 CPU/PCIe switch
- NUMA
- 多条 collective 是否争用同一链路
- rank mapping 是否合理

面试中不要只说 “机器有 NVLink”。要说明：

- 哪些 GPU 直接连接
- 是否有 NVSwitch
- 跨节点 NIC 数量
- TP、PP、DP process group 如何映射到快慢链路

通常高频、细粒度的 TP 通信优先放在高速节点内；DP 可以容忍相对低频的跨节点通信，但最终仍需结合模型与机器拓扑。

6.5 五、通信与计算重叠

重叠成立需要：

1. 当前通信依赖的数据已经生成。
2. 后续存在不依赖该通信结果的计算。
3. 使用异步 collective 和独立 stream。
4. 正确设置 event/stream 依赖。
5. GPU、copy engine、网络和 SM 资源没有严重争用。

常见方法：

- Gradient bucketing：一部分梯度 ready 后立即发起通信。
- 分块 collective：减小等待全部 tensor ready 的时间。
- Pipeline parallel 交错 microbatch。
- Ring/Context Attention 在计算当前 block 时预取下一块 K/V。
- Double buffering。

为什么 “异步发起” 不等于 “成功隐藏”：

- 通信可能与 kernel 竞争 SM 或 HBM 带宽。

- bucket 太小导致 latency 开销过大。
- bucket 太大导致发起太晚。
- 错误同步会把两个 stream 重新串行化。
- critical path 上最终仍要等待。

验证重叠要看 Nsight Systems 时间线，而不是只看代码使用了 async_op。

6.6 六、并行策略对比

并行方式	切分对象	主要通信	主要目的
DP	batch	梯度 All-Reduce/RS	扩大吞吐
TP	权重和 hidden/head	每层 AG/AR/RS	单层模型切分
PP	layers	stage 间 send/recv	模型深度切分
SP	sequence activation	AG/RS 或 A2A	降 activation/支持长序列
CP/Ring	context	K/V P2P 或相关 collective	超长上下文
EP	experts	token All-to-All	MoE expert 切分

组合并行时，device 数通常来自多个维度的乘积，但具体框架可能复用 process group：

$$\text{World Size} = DP \times PP \times TP \times \text{其他独立并行度}$$

经典 Megatron SP 通常复用 TP group，因此不一定作为独立乘数。

6.7 七、扩展效率

Strong Scaling：

- 固定总问题规模，增加 GPU。
- 每卡计算减少，通信比例通常上升。

$$\text{Efficiency_strong} = T_1 / (p \times T_p)$$

Weak Scaling：

- 每卡问题规模固定，GPU 增加时总规模增加。
- 理想情况下 step time 不变。

$$\text{Efficiency_weak} = T_1 / T_p$$

常见扩展瓶颈：

- 每卡 GEMM 变小，计算效率下降。
- collective latency 和带宽占比增加。
- 跨节点链路成为瓶颈。
- load imbalance。

- pipeline bubble。
- CPU dataloader 或调度跟不上。

6.8 八、性能指标

6.8.1 MFU

Model FLOPs Utilization：

> MFU = 实际每秒完成的模型理论 FLOPs / 硬件理论峰值 FLOPs

要说明：

- 模型 FLOPs 如何估算。
- 使用哪种精度对应的硬件峰值。
- 是否把重计算、padding、通信算入分子。

6.8.2 HFU

Hardware FLOPs Utilization 往往会把实际执行的额外计算也计入，因此 activation recomputation 场景下 HFU 可能高于 MFU。

6.8.3 吞吐与延迟

- Training：samples/s、tokens/s、step time。
- Serving：TTFT、TPOT、tokens/s、P50/P99。
- 不同 batch/sequence 下指标不可直接比较。

6.9 九、典型面试题

6.9.1 All-Reduce 为什么能拆成 Reduce-Scatter + All-Gather?

Reduce-Scatter 完成逐元素归约并让每卡保留一段最终结果；All-Gather 再交换各段，使每卡获得完整归约结果。

6.9.2 TP 为什么更适合放在节点内?

TP 通信发生在每层，频率高且处于关键路径，对延迟和带宽敏感；NVLink/NVSwitch 通常比跨节点网络更适合。

6.9.3　通信量不变，为什么性能仍可能变差?

消息变碎导致 α 项增加；拓扑或 rank mapping 变差；多通信争用链路；计算变小导致无法隐藏；collective 算法变化。

6.9.4　如何判断 NCCL 是真正瓶颈?

看端到端关键路径、通信与计算是否重叠、等待时间和扩展曲线；再通过关闭重叠、改变 bucket、改变 rank mapping 或单机/多机对比做实验。

6.9.5　All-to-All 为什么难优化?

每个 rank 同时与多个 peer 交换不同数据，容易受流量不均、网络拥塞、拓扑和小消息影响；MoE 还存在 token 数不均衡。

6.10　十、现场计算练习

1. bf16 tensor shape 为 [8,4096,8192]，计算 tensor 大小。
2. 在 8 卡 Ring All-Reduce 下，按发送字节口径计算每卡流量。
3. 假设有效带宽为 200 GB/s，忽略 latency，估算通信下限。
4. 加入固定 10 μs 的每轮 latency，讨论小 tensor 的影响。
5. 对比单机 NVLink 和跨节点链路，判断 TP group 如何映射。

第 1 题答案：

$8 \times 4096 \times 8192 \times 2$ bytes = 536,870,912 bytes，约 512 MiB。

第 2 题答案：

$2 \times 7/8 \times 512$ MiB = 896 MiB 每卡发送量。

后续题要注明这里只是理想带宽模型，真实耗时还受算法、链路、同步和竞争影响。

6.11　官方资料

- NCCL User Guide：https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/
- CUDA Best Practices Guide：https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/
- Megatron Core Developer Guide：https://docs.nvidia.com/megatron-core/developer-guide/latest/

第七章　Vibe Coding 回答与技巧

7.1　一、推荐的面试回答

7.1.1　30 秒版

我把 Vibe Coding 当作工程加速器，而不是正确性保证者。它很适合生成样板代码、测试、benchmark、调试脚本和探索性实现。我的习惯是先明确接口、约束和验收标准，让模型做小步修改；每一步都审查 diff，并运行测试和性能分析。对于并行语义、数值稳定性、性能关键路径和安全边界，我不会直接接受生成结果，而会自己推导和验证。

7.1.2　2 分钟版

我通常把流程分成五步。第一步是先写目标和验收标准，包括输入输出、shape、精度、硬件与性能目标；第二步让模型先阅读相关代码并复述约束，不马上改代码；第三步把任务拆成小改动，限制文件范围，并尽可能先生成测试；第四步检查 diff、编译、单测、端到端结果和 benchmark；第五步用 profiler 或日志验证收益机制，而不是只看代码能运行。

在 GPU kernel 场景中，我会特别检查 layout、mask、同步、数值累加、非整齐 shape、异步计时和性能回退。AI 可以快速提供候选实现，但最后对正确性和性能数据负责的仍然是工程师。

7.2　二、适合使用的场景

- 生成 boilerplate、配置和接口胶水。
- 补单元测试、边界用例和 benchmark harness。
- 快速解释陌生模块的数据流。
- 将 profiler 结果整理成假设列表。
- 生成多个候选 kernel 配置。
- 编写数据处理、日志分析和可视化脚本。
- 重构前识别调用点和影响范围。
- 起草文档与 code review checklist。

7.3 三、不应直接托管给模型的场景

- 未经检查的生产数据删除或迁移。
- 密钥、访问令牌和敏感数据处理。
- 分布式同步、并发和内存生命周期的关键逻辑。
- 没有 oracle 的数值算法修改。
- 只在单一 shape 上看起来更快的 GPU kernel。
- 大范围重构后一次性合并。
- 模型声称 “测试通过” 但没有真实命令输出。

7.4 四、可靠工作流

7.4.1 1. 先写清契约

提供：

- 目标
- 非目标
- 输入输出
- shape 和 dtype
- 错误处理
- 可改文件范围
- 正确性阈值
- 性能指标
- 必须运行的验证

7.4.2 2. 先理解再修改

推荐提示词：

> 先阅读这些文件，说明调用链、数据 shape、现有约束和你不确定的地方。现在不要改代码。

这样能提前发现模型是否误解上下文。

7.4.3 3. 小步修改

- 一次解决一个问题。
- 每步保持 diff 可审查。
- 不混入无关重构。
- 每个逻辑步骤留一个可回退点。

7.4.4 4. 测试先行

推荐提示词：

> 先写能暴露当前问题的最小测试，并运行确认它会失败；再提出最小修改。

对 kernel 至少覆盖：

- 普通 shape
- 非整除 shape
- 极小 shape
- 不同 dtype
- 连续/非连续输入
- 极值和数值稳定性

7.4.5 5. 证据式验收

要求模型提供：

- 修改了什么
- 为什么
- 运行了什么命令
- 关键输出
- 未覆盖的风险
- benchmark 条件和结果

但最终仍要自己看真实日志和代码。

7.5 五、GPU/Triton 专用检查表

□ 所有 pointer arithmetic 与 tensor stride 一致
□ 边界都有正确 mask
□ padding 值不会污染 reduction
□ reduction 使用合理累加精度
□ causal mask 与 padding mask 没混淆
□ 没有越界 load/store
□ GPU 异步计时方式正确
□ baseline 语义、dtype、layout 一致
□ 多个真实 shape 都测过
□ 没有因寄存器溢出导致性能下降
□ profiler 证实优化机制

7.6 六、好的提示词结构

任务：
优化一个 [B,S,H] 的 RMSNorm Triton kernel。

约束：
- 输入 bf16，fp32 累加
- H 可能不是 2 的幂
- 不修改调用接口
- 先只做 forward

验收：
- 与 PyTorch 参考实现 max error < ...
- 覆盖 H=768/4096/8192 和非整齐 shape
- 给出 warmup、计时和同步方法
- 不允许只报告单个 shape

工作方式：
1. 先复述数据布局和风险，不改代码。
2. 给出两种实现及代价。
3. 经确认后做最小实现。
4. 运行测试和 benchmark。
5. 汇报证据与剩余风险。

7.7 七、常见失败模式

7.7.1 上下文不足

表现：生成不存在的接口、忽略项目约束。

解决：提供入口文件、调用链、接口和相关测试；先让模型复述理解。

7.7.2 一次修改太大

表现：diff 难审查，问题无法定位。

解决：拆成测试、最小实现、优化、清理四步。

7.7.3 过度相信输出

表现：引用虚构 API、声称 benchmark 成功。

解决：要求真实执行；自己核对文档、日志和代码。

7.7.4 只追求代码能跑

表现：边界 shape 错误，性能甚至更差。

解决：把 correctness matrix、benchmark matrix 和 profiler 指标写入验收条件。

7.7.5 不管理版本

表现：错误修改难以恢复。

解决：小步 commit、独立分支或 worktree，并保持工作区状态清晰。

7.8 八、加分表达

可以主动说明三个层次：

1. **生成速度**：AI 提升初稿、搜索和样板代码速度。
2. **验证成本**：生成越快，验证环节越要制度化。
3. **责任边界**：工程师对接口、正确性、性能、安全和上线负责。

避免两种极端回答：

- “基本都让 AI 写，我只看结果。”
- “AI 完全没有用，我不用。”

成熟回答应体现：会用、会约束、会验证，也知道何时不用。

第八章　模拟面试题与评分表

8.1　一、45 分钟模拟流程

时间	环节
0～3 分钟	自我介绍与方向概括
3～15 分钟	项目深挖
15～23 分钟	GPU 架构与性能
23～32 分钟	TP/SP Attention 白板推导
32～38 分钟	Triton
38～42 分钟	通信与扩展
42～45 分钟	Vibe Coding 与反问

全程录音。结束后先自己评分，再查材料。

8.2　二、项目题

1. 用一分钟介绍你最重要的项目。
2. 这个项目的 baseline 是什么？
3. 你如何确认主要瓶颈？
4. 你个人负责的边界在哪里？
5. 为什么选择当前方案？
6. 哪个方案尝试后失败了？为什么？
7. 你的性能数字是在什么硬件和 shape 下测的？
8. 如何保证 benchmark 公平？
9. 如何验证数值正确性？
10. 如果 GPU 数量翻倍，最先出现的瓶颈是什么？
11. 如果 sequence length 翻倍，计算、显存、通信分别如何变化？
12. 如果重新做一次，你会先改变什么？

8.3 三、GPU 架构题

1. 一个 CUDA block 如何被调度到 SM?
2. SIMT 与 SIMD 有什么区别?
3. 什么是 warp divergence?
4. 什么是 memory coalescing?
5. shared-memory bank conflict 如何产生?
6. register pressure 为什么会影响 occupancy?
7. occupancy 越高越好吗?
8. Tensor Core 与 CUDA Core 有什么区别?
9. 什么情况下 kernel 是 memory-bound?
10. 如何使用 Roofline Model?
11. 为什么 matmul 适合 tiling?
12. kernel fusion 为什么会加速，也可能为什么变慢?
13. FlashAttention 为什么更快?
14. Nsight Systems 与 Nsight Compute 分别解决什么问题?
15. 如何判断一个优化是否只是测量误差?

8.4 四、TP/SP 白板题

统一先给：

B=8，S=4096，H=8192，A=64，D=128，TP=8

问题：

1. 写出单卡 MHA 全部 tensor shape。
2. QKV 权重如何切，为什么是 column parallel?
3. 每卡有多少 heads?
4. Score shape 是什么?
5. O projection 如何切?
6. 哪一步需要 All-Reduce，为什么不能 concat?
7. backward 哪一步还需要通信?
8. All-Reduce 的逻辑 tensor 多大?
9. bf16 下字节数是多少?
10. Ring All-Reduce 每卡发送量是多少?
11. 加 Megatron SP 后，层间 activation shape 是什么?
12. QKV 前发生什么通信?
13. O projection 后发生什么通信?
14. 为什么经典 SP 没有消除 S^2 score?
15. Ulysses 的 All-to-All 转换哪两个切分维度?
16. Ring Attention 如何避免聚合完整 K/V?
17. GQA 中 KV heads 不能被 TP 整除怎么办?
18. Decode 时 KV cache 如何切?

8.5　五、Triton 题

1. 一个 Triton program 与 CUDA block 的关系是什么?
2. tl.program_id 和 tl.arange 分别做什么?
3. 为什么边界需要 mask?
4. 写出 row-wise softmax。
5. softmax 为什么要减 max?
6. padding lane 为什么用负无穷?
7. matmul 的 BLOCK_M/N/K 如何选择?
8. num_warps 和 num_stages 有什么影响?
9. 为什么更大的 tile 不一定更快?
10. 如何计算 vector add 的有效带宽?
11. 如何做可靠 GPU benchmark?
12. 如何检查 register spilling?
13. 如何设计 autotune 搜索空间?
14. 一个融合 kernel 变慢，如何排查?
15. 如何验证非整齐 shape?

8.6　六、分布式通信题

1. 解释 All-Reduce、All-Gather、Reduce-Scatter、All-to-All。
2. 为什么 All-Reduce 可以拆成 RS+AG?
3. Ring 和 Tree 算法各适合什么场景?
4. α-β 模型是什么?
5. 为什么 TP 通常优先放节点内?
6. 通信异步发起就一定能被隐藏吗?
7. 如何用 timeline 证明通信计算重叠?
8. bucket 太大或太小分别有什么问题?
9. strong scaling efficiency 如何计算?
10. 多机性能突然下降，先检查什么?

8.7　七、Vibe Coding 题

1. 你怎么使用 AI 编程?
2. 哪些任务适合交给 AI?
3. 如何避免生成错误 API?
4. 如何验证 GPU kernel?
5. AI 给出的代码 benchmark 很快，你会检查什么?
6. 如何保护敏感数据?
7. 如何控制大规模修改的风险?

8.8 八、评分表

每项 0～4 分：

分数	标准
0	完全不会
1	只记得名词
2	能解释概念，但推导或证据不足
3	能完整回答，有公式、shape 或数据
4	能回答追问、边界、代价和替代方案

维度	权重	得分
项目叙事与个人贡献	20	
瓶颈分析与量化证据	15	
GPU 架构	15	
TP/SP shape 推导	20	
通信与拓扑	10	
Triton	15	
Vibe Coding 与工程习惯	5	

总分换算：

$$\text{加权得分} = \Sigma(\text{单项得分} / 4 \times \text{权重})$$

- 85～100：可以应对连续深挖。
- 70～84：主体可靠，补最弱的两项。
- 55～69：概念知道，但白板和量化不足。
- 低于 55：先集中完成 P0 内容。

8.9 九、复盘模板

每轮模拟后填写：

- 最卡顿的问题：
- 说错的 shape/公式：
- 缺少的项目数字：
- 含糊使用的术语：
- 最长的一段无效表达：
- 一个回答得好的地方：
- 明天优先补的两个问题：

8.10　十、适合向面试官反问

1. 团队目前更关注训练、推理还是通信库/算子层优化？
2. 主要硬件平台、集群规模和互联拓扑是什么？
3. 性能工作的评价指标更偏单算子、端到端吞吐还是扩展效率？
4. Triton、CUDA 和框架级优化在团队中的工作比例如何？
5. 当前最希望新同学解决的性能瓶颈是什么？

反问应帮助判断实际工作内容，不要问官网即可找到的信息。

第九章　面试前一页速查

9.1　项目回答

背景与规模 → 我的职责 → baseline → profiler 证据 → 核心方案 → 量化结果 → 正确性 → 失败与反思

必备数字：

- GPU 型号/数量/互联
- B/S/H/dtype
- 优化前后 latency、throughput、显存、MFU
- benchmark 方法
- 通信占比

9.2　GPU

- Warp：通常 32 threads。
- Coalescing：相邻线程访问连续、对齐地址。
- Bank conflict：同 warp 访问同 bank 的不同地址导致串行化。
- Occupancy：活跃 warp 比例，是隐藏 latency 的手段，不是目标。
- Roofline：Perf ⩽ min(Peak FLOPS, AI × Bandwidth)。
- Fusion：减少 HBM 往返和 launch，但可能增加 register pressure。
- FlashAttention：分块 + online softmax，避免 S×S score 落 HBM。

9.3　TP Attention

符号：

X=[B,S,H]，A heads，D=H/A，TP=T

每卡：

W_qkv=[H,3H/T]

Q/K/V=[B,A/T,S,D]

Score=[B,A/T,S,S]

Context=[B,S,H/T]

W_o=[H/T,H]

Y_partial=[B,S,H]

Y=AllReduce_sum(Y_partial)

反向：QKV column-parallel 对 dX 的 partial 需要求和。

9.4 Megatron SP

X_sp=[B,S/T,H]

All-Gather(sequence) → [B,S,H]

本地 TP Attention

O projection partial=[B,S,H]

Reduce-Scatter(sequence) → [B,S/T,H]

经典 SP 节省 LayerNorm/residual/dropout 等 activation，不直接切小 S^2 score。

问清楚 SP 定义：

- Megatron SP：AG + RS。
- Ulysses：sequence ↔ head 的 All-to-All。
- Ring/Context Parallel：环形传 K/V + online softmax。

9.5 Collective

- All-Reduce：每卡得到完整求和结果。
- All-Gather：分片拼成完整结果。
- Reduce-Scatter：求和后每卡保留一片。
- All-to-All：每卡向所有 peer 交换不同分片。

Ring，逻辑 tensor N bytes：

- AG：约 (p-1)/p × N 每卡发送。
- RS：约 (p-1)/p × N。
- AR：约 2(p-1)/p × N。

9.6 Triton

program_id 定位 tile → arange 生成 offsets → mask 边界 → load → compute/reduce → store

调优：

- BLOCK size
- num_warps
- num_stages
- layout/coalescing
- register/shared-memory pressure
- autotune

Benchmark：

先正确性；warmup；正确同步；多 shape；相同语义 baseline；报告 median + GB/s/TFLOPS。

9.7 Vibe Coding

AI 是工程加速器，不是正确性保证者。

工作流：

明确契约 → 先读代码复述约束 → 小步修改 → 测试先行 → 看 diff → 真实执行 → benchmark/profiler → 汇报风险

GPU 代码重点检查：

- shape/stride
- mask/越界
- 累加精度
- 同步和异步计时
- 非整齐 shape
- 多 shape 性能

9.8 现场推导规则

1. 定义符号和 layout。
2. 说明 MHA/GQA/MQA。
3. 说明切输入维还是输出维。
4. 逐层写 input × weight → output。
5. 判断是 shard、replica 还是 partial sum。
6. 需要拼接、求和、换布局时才放 collective。
7. 补 backward、通信量和边界条件。

9.9　最后提醒

- 不知道定义时先确认，不要猜。
- 所有性能数字带测试条件。
- 不把异步发起等同于成功重叠。
- 不把高 occupancy 等同于高性能。
- 不把能运行等同于正确且高效。

腾讯 HPC 面试 QA 知识手册

60 个核心问答・标准回答・追问・易错点

面试备战资料

2026-10-21

目录

第一章 使用说明 **3**

第二章 GPU 架构与性能分析 **4**

2.1 执行模型 4

2.2 存储体系 5

2.3 计算与性能模型 7

第三章 Triton 编程与调优 **10**

3.1 编程模型 10

3.2 Reduction 与 Softmax 11

3.3 Matmul 与自动调优 12

第四章 Attention、Tensor Parallel 与 Sequence Parallel **14**

4.1 单卡 Attention 14

4.2 Tensor Parallel 15

4.3 Sequence 与 Context Parallel 17

第五章 通信与分布式高性能计算 **20**

5.1 Collective 与性能模型 20

5.2 拓扑、重叠与扩展 21

第六章 项目深挖与 Vibe Coding **24**

6.1 项目表达 24

6.2 Vibe Coding 25

第一章　使用说明

本手册采用统一的面试 QA 结构：

- **Q：**面试官可能提出的问题。
- **标准回答：**先给结论，再说明原因和条件。
- **可能追问：**用来继续向下准备。
- **易错点：**面试中常见的概念混淆。

建议先遮住答案口述，再阅读标准回答。能复述不等于掌握；涉及张量和通信的题目必须在白板上推导。

第二章　GPU 架构与性能分析

2.1　执行模型

2.1.1　QH-G01 ｜ GPU 的硬件和编程层级是什么?

Q：请从 GPU 到 thread 说明执行层级。

标准回答：

硬件上可以概括为 GPU → GPC → SM → warp scheduler 与执行单元，执行单元包括 CUDA Core、Tensor Core、load/store unit 等。编程模型是 grid → block/CTA → warp → thread。一个 block 被放到某个 SM 后，通常在该 SM 上运行到结束；block 内线程以 warp 为基本调度单位，一个 warp 通常包含 32 个线程。

可能追问：一个 SM 能同时驻留多少个 block？受什么限制?

易错点： CUDA Core 数量不等于同时执行的 thread 数；不同架构的具体单元数量要查目标型号。

2.1.2　QH-G02 ｜ SIMT 和 SIMD 有什么区别?

Q： GPU 为什么称为 SIMT，而不只是 SIMD?

标准回答：

SIMD 强调一条向量指令显式操作多个数据 lane；SIMT 把每个 lane 暴露为独立 thread，每个 thread 有自己的寄存器、地址和控制状态，但同一 warp 的线程共享指令发射。SIMT 让开发者以线程方式表达并行，同时硬件以 warp 批量执行。

可能追问：分支和独立线程状态如何同时存在?

易错点： SIMT 不代表同一 warp 的线程可以在同一周期任意执行不同指令。

2.1.3　QH-G03 ｜什么是 warp divergence?

Q：一个 if 分支为什么可能降低 GPU 性能?

标准回答：

如果同一 warp 的线程对分支条件结果不同，硬件需要分别执行各条有效路径，并用 active mask 屏蔽不属于当前路径的线程，因此有效并行度下降。若分支在 warp 间不同、但同一 warp 内一致，则通常没有这种路径串行化问题；很短的分支也可能被编译器谓词化。

可能追问：如何通过数据重排减少 divergence?

易错点：不能看到代码中有 if 就断言一定很慢。

2.1.4 QH-G04 | GPU 如何隐藏延迟?

Q： HBM 延迟很高，GPU 为什么仍能保持高吞吐?

标准回答：

当一个 warp 因数据依赖或访存等待而不可执行时，warp scheduler 可以切换到其他 ready warp。足够的并行 warp、独立指令、异步数据搬运和流水化共同隐藏延迟。隐藏延迟的前提是有可执行工作，并且资源没有被寄存器或 shared memory 用量限制得过紧。

可能追问： occupancy 高为什么仍可能无法隐藏延迟?

易错点： latency hiding 不等于降低单次访存延迟。

2.2 存储体系

2.2.1 QH-G05 | GPU memory hierarchy 如何组织?

Q：请比较 register、shared memory、L1、L2 和 HBM。

标准回答：

Register 通常是线程私有、延迟最低，但容量有限；shared memory 由 block 内线程共享并由程序显式管理；L1 位于 SM 附近，L2 为全 GPU 共享 cache；HBM 容量和带宽高，但延迟显著高于片上存储。优化目标是提高局部性和复用，让同一份 HBM 数据在片上被使用多次。

可能追问： L1 与 shared memory 的容量关系是否固定?

易错点：不要给所有架构套用固定延迟数字或固定容量划分。

2.2.2 QH-G06 | 什么是 memory coalescing?

Q：为什么同一 warp 的地址模式会影响 global-memory 性能?

标准回答：

当同一 warp 的线程访问连续、对齐且可合并的地址时，硬件可用较少的 memory transaction 完成访问。大 stride、散乱地址或不对齐会增加 transaction 数量和无效字节。常见优化包括改变 layout、让连续线程访问连续元素、向量化 load/store，以及用 shared-memory tile 完成转置。

可能追问： AoS 和 SoA 哪个更利于 coalescing?

易错点： “地址连续” 要从一个 warp 的整体访问模式判断，而不是只看单个线程。

2.2.3 QH-G07 | 什么是 shared-memory bank conflict?

Q： shared memory 已经在片上，为什么仍可能访问变慢?

标准回答：

Shared memory 被划分为多个 bank。同一 warp 的多个线程若在同一时刻访问同一个 bank 的不同地址，请求可能被拆成多次完成。常见解决方式是改变索引映射，或给二维 tile 增加 padding；访问同一地址的合法 broadcast 通常不属于普通 bank conflict。

可能追问：为什么转置 tile 常从 N×N 改为 N×(N+1)?

易错点： bank 数和 bank width 与架构有关，不要脱离目标 GPU 死背。

2.2.4 QH-G08 | register pressure 如何影响性能?

Q：一个 kernel 使用的寄存器越多会发生什么?

标准回答：

每个 SM 的物理寄存器总量有限。每线程寄存器增加会减少可同时驻留的 warps 或 blocks，降低 occupancy；如果超过可分配范围，还可能 spill 到 local memory，产生额外访存。不过更多寄存器也可能减少重复计算和内存访问，所以目标不是寄存器越少越好，而是总体性能最优。

可能追问：如何发现 register spilling?

易错点： local memory 名字中有 local，但通常位于设备内存层次，不等于片上寄存器。

2.2.5 QH-G09 | occupancy 越高越好吗?

Q：为什么 100% occupancy 不一定最快?

标准回答：

Occupancy 是活跃 warp 数相对于架构上限的比例，它主要帮助隐藏延迟。当 kernel 已经 compute-bound，或受指令依赖、访存不合并、cache miss、带宽限制时，提高 occupancy 不一定有收益；为了追求 occupancy 缩小 tile，还可能降低数据复用。应结合吞吐、stall reason、寄存器、shared memory 和 Roofline 判断。

可能追问： occupancy 下降但性能提高有哪些可能原因?

易错点： Occupancy 是手段，不是性能 KPI。

2.3 计算与性能模型

2.3.1 QH-G10 | CUDA Core 和 Tensor Core 有什么区别?

Q：大模型为什么强调 Tensor Core 利用率?

标准回答：

CUDA Core 处理通用标量或向量算术；Tensor Core 面向小矩阵块的乘加，以较低精度输入和较高精度累加提供更高吞吐。大型 GEMM 被分解成 block tile、warp tile 和指令 tile。能否高效使用 Tensor Core 取决于 dtype、shape、layout、对齐和实现。

可能追问： fp16/bf16 输入为什么常用 fp32 accumulator?

易错点：使用低精度 dtype 不代表一定进入 Tensor Core 路径。

2.3.2 QH-G11 | 什么是 Roofline Model?

Q：如何用 Roofline 判断性能上限?

标准回答：

算术强度 AI = FLOPs / Bytes。性能上限可写为：

$$\text{Performance} \leqslant \min(\text{Peak Compute}, \text{AI} \times \text{Memory Bandwidth})$$

AI 较低时更可能受带宽限制；AI 较高并接近计算峰值时更可能受算力限制。分析时必须说明 Bytes 对应 HBM、L2 还是其他存储层，并用 profiler 和控制变量实验验证。

可能追问：实际性能远低于两条 roof 时应检查什么?

易错点： Roofline 给出上限，不自动解释所有 launch、同步、依赖和分支问题。

2.3.3 QH-G12 | 为什么 GEMM 要做 tiling?

Q： Tiling 如何提高 GEMM 性能?

标准回答：

一个 A tile 和 B tile 从较慢存储加载后，可被多个输出元素重复使用。这样 HBM bytes 的增长慢于 FLOPs，提高 arithmetic intensity。实际实现还通过寄存器、shared memory、异步加载和双缓冲建立多级 tile 与流水。

可能追问： tile 为什么不能无限增大?

易错点：大 tile 会增加寄存器、shared memory、尾块浪费并降低可驻留并行度。

2.3.4 QH-G13 | kernel fusion 为什么可能更快，也可能更慢?

Q：如何评价一个 fusion 方案?

标准回答：

Fusion 可减少中间 tensor 的 HBM 写回与读入、kernel launch 和同步，因此常对 memory-bound 的算子链有效。但融合会延长 live range、增加寄存器和 shared-memory 压力、降低 occupancy，并可能破坏库 kernel 的高效实现。应比较减少的 Bytes/launch 与新增的资源和调度代价。

可能追问：如何用 profiler 证明收益来自 HBM I/O 减少?

易错点：不要把 “kernel 数量减少” 直接等同于端到端一定加速。

2.3.5 QH-G14 | FlashAttention 为什么更快?

Q： FlashAttention 的核心优化是什么?

标准回答：

标准实现会将 S×S score 和 softmax 中间结果多次写入、读回 HBM。FlashAttention 对 Q/K/V 分块，在片上存储中计算，并用 online softmax 合并各 K/V block 的统计量，从而避免完整 S×S 中间矩阵落到 HBM。它主要减少 I/O 和 activation 显存，标准版本并没有消除 Attention 的主项 FLOPs。

可能追问： Online softmax 需要维护哪些状态?

易错点： FlashAttention 是精确 Attention 算法，不等于稀疏或近似 Attention。

2.3.6 QH-G15 | 如何系统定位 GPU 性能瓶颈?

Q：从端到端到单 kernel 的分析顺序是什么?

标准回答：

先用 Nsight Systems 或框架 profiler 看端到端时间线，识别 CPU launch、GPU 空洞、热点 kernel、NCCL 和同步；再用 Nsight Compute 检查热点 kernel 的 FLOPs、带宽、occupancy、寄存器、shared memory、stall reason 和访存效率；最后结合 Roofline 和控制变量实验验证因果。

可能追问： Nsight Systems 与 Nsight Compute 为什么不能互相替代?

易错点：单个利用率指标不足以证明根因。

2.3.7 QH-G16 | GPU benchmark 怎样才可信?

Q：为什么直接用 CPU wall clock 包住一次 kernel 不可靠?

标准回答：

GPU 默认异步执行，CPU 返回不表示 kernel 已完成。应先 warmup，使用 CUDA Event 或在计时边界正确同步，多次重复并报告 median/分位数。Baseline 必须采用相同语义、dtype、layout、shape 和算法条件，同时先验证正确性，再报告 latency、GB/s 或 TFLOPS。

可能追问：为什么首次调用不能计入稳定态耗时？

易错点：编译、autotune、cache 冷启动和 lazy initialization 会污染首次测量。

第三章　Triton 编程与调优

3.1　编程模型

3.1.1　QH-T01 ｜ Triton program 与 CUDA block 有什么关系？

Q： Triton 的核心编程抽象是什么？

标准回答：

Triton 以 program instance 描述一个数据 tile 的向量化计算。开发者用 program_id 确定 tile，用 arange 构造 tile 内 offsets，再以向量方式 load、compute、store。编译器负责把这种描述映射到 GPU 线程和指令。它通常可类比 CUDA block，但不是要求开发者逐线程书写的完全等价抽象。

可能追问：哪些硬件问题仍需开发者负责？

易错点： Triton 不会自动替你选择所有最佳 tile、layout 和资源配置。

3.1.2　QH-T02 ｜ program_id、arange 和 mask 分别做什么？

Q：如何定位一个 Triton program 负责的数据？

标准回答：

program_id 定位当前 program 的全局 tile；arange 生成 tile 内的向量 offsets；二者组合得到实际元素地址。若总长度不是 block size 的整数倍，mask 标记有效 lane，load 和 store 必须使用它避免越界；reduction 的无效 lane 还需要选择不污染结果的 other 值。

可能追问： Softmax 的 padding lane 应填什么？

易错点：只给 store 加 mask 不一定安全，越界 load 同样是错误。

3.1.3　QH-T03 ｜ Triton 相比 CUDA 抽象了什么？

Q：为什么 Triton 更容易编写高性能 kernel？

标准回答：

Triton 用 block 级向量程序代替显式 thread 编排，编译器承担一部分线程映射、访存合并和指令生成工作。开发者仍需决定 tile、数据布局、并行维度、reduction、num_warps、num_stages 和算法结构。因此它降低了实现门槛，但没有消除硬件性能推理。

可能追问：哪些场景 CUDA 仍更合适?

易错点：不要把抽象层更高理解为所有场景都比 CUDA 快。

3.2 Reduction 与 Softmax

3.2.1 QH-T04 ｜如何实现稳定的 fused softmax?

Q： Row-wise softmax 的 Triton 核心步骤是什么?

标准回答：

一个基础实现可让每个 program 处理一行：

```
offsets = arange(0, BLOCK_SIZE)
x = load(row + offsets, mask, other=-inf)
x = x - max(x)
p = exp(x)
y = p / sum(p)
store(y, mask)
```

减最大值避免 exp 溢出；无效 lane 填负无穷，使其 exp 为 0；BLOCK_SIZE 常取覆盖 n_cols 的 2 的幂以利 reduction。

可能追问： n_cols 很大时为什么不能继续用单 program?

易错点： Mask 与数值稳定性是两个不同问题。

3.2.2 QH-T05 ｜为什么 reduction 容易带来资源压力?

Q：行宽增加后 softmax 为什么可能突然变慢?

标准回答：

Program 需要保存较宽的输入和中间向量，并进行跨 lane reduction。行宽增大可能显著增加寄存器需求，降低 occupancy 或触发 spilling；单个 program 的并行映射也可能达到不合适范围。解决方法包括分块、多阶段 reduction、重新设计数据流或使用专用算法。

可能追问：如何通过编译信息和 profiler 证明 spilling?

易错点：把 BLOCK_SIZE 调大并不总能换来更高吞吐。

3.3 Matmul 与自动调优

3.3.1 QH-T06 | Triton matmul 如何做 tiling?

Q： BLOCK_M、BLOCK_N、BLOCK_K 分别代表什么?

标准回答：

每个 program 负责 C 的 BLOCK_M×BLOCK_N tile，沿 K 维以 BLOCK_K 为步长循环。每轮加载 A 的 BLOCK_M×BLOCK_K 和 B 的 BLOCK_K×BLOCK_N，使用 tl.dot 累加到通常为 fp32 的 accumulator，最后转换并写回输出。

可能追问：尾部 M/N/K 如何处理?

易错点：权重 layout、stride 和是否能走 Tensor Core 路径同样关键。

3.3.2 QH-T07 | num_warps 和 num_stages 有什么作用?

Q：这两个参数为什么会改变性能?

标准回答：

num_warps 控制一个 program 使用多少 warp 协作执行，影响并行度和资源分配；num_stages 控制软件流水或异步加载的阶段深度，有助于隐藏加载延迟，但会增加 shared memory 和在途数据资源。最佳值依赖 tile、dtype、shape 和架构。

可能追问： stages 越多为什么可能更慢?

易错点：参数不能脱离具体 tile 单独判断。

3.3.3 QH-T08 | Autotune 应如何设计?

Q：为什么不能盲目枚举所有配置?

标准回答：

候选应覆盖合理的 BLOCK_M/N/K、num_warps 和 num_stages 组合，并排除明显超出资源或不适合 shape 的配置。Autotune key 选择真正会改变最佳配置的维度或 shape 分段。首次编译和搜索成本不能计入稳定态 benchmark，动态 shape 太多还会导致配置与缓存膨胀。

可能追问：如何防止只对单个 shape 过拟合?

易错点： Autotune 找到局部最快配置，不代表算法层面已经最优。

3.3.4 QH-T09 |如何验证 Triton kernel 的正确性?

Q：只与 PyTorch 比一次 allclose 够吗?

标准回答：

至少覆盖典型、极小、极大、非整除 shape，不同 dtype，连续与非连续输入，以及极值和数值稳定性场景；明确 atol/rtol 和参考实现。Reduction 类算子要检查累加精度，Attention 要区分 causal 与 padding mask。还需要端到端回归，不能只验证单 kernel。

可能追问： fp16、bf16 的误差阈值如何定？

易错点： Allclose 通过不代表语义、布局或梯度一定正确。

3.3.5 QH-T10 | Triton kernel 比 baseline 慢时怎么排查？

Q：给出一个系统调优顺序。

标准回答：

先排除正确性和计时错误；计算理论 FLOPs 与 Bytes，判断大致瓶颈；检查访问是否连续、对齐和重复；检查 tile、尾块利用率、num_warps、num_stages；查看寄存器、shared memory、occupancy 和 spilling；最后在多个真实 shape 上 autotune，并用 profiler 验证收益机制。

可能追问：小 shape 下为什么很难超过高度优化库？

易错点：不要在错误 baseline 或单个有利 shape 上宣布优化成功。

第四章　Attention、Tensor Parallel 与 Sequence Parallel

4.1　单卡 Attention

4.1.1　QH-A01｜标准 MHA 的 tensor shape 如何变化?

Q：输入为 [B,S,H] 时，从 X 到 Y 的完整 shape 是什么?

标准回答：

设 A 为 head 数，D=H/A，权重使用 [in_dim,out_dim]：

X [B,S,H]

W_qkv [H,3H] → QKV [B,S,3H]

Q/K/V [B,A,S,D]

Score=QK^T [B,A,S,S]

Prob [B,A,S,S]

Context=Prob×V [B,A,S,D] → [B,S,H]

W_o [H,H] → Y [B,S,H]

可能追问：如果使用 [S,B,H] layout，哪些维度会变化?

易错点： Score 有两个 sequence 维。

4.1.2　QH-A02 ｜ Attention 的 FLOPs 如何估算?

Q： Projection 和 Attention core 的主项复杂度是什么?

标准回答：

按一次乘加计 2 FLOPs，QKV projection 约 $6BSH^2$，output projection 约 $2BSH^2$；QK^T 约 $2BS^2H$，Prob×V 约 $2BS^2H$，所以 Attention core 合计约 $4BS^2H$。短序列、大 hidden 时 projection 占比可能更高；长序列时 S^2 项增长更快。

可能追问： FlashAttention 改变这些主项 FLOPs 吗?

易错点：要说明是否包含 backward、causal 有效区域和重计算。

4.2 Tensor Parallel

4.2.1 QH-A03 | TP 下 QKV 为什么做 column parallel?

Q：每卡的 QKV 权重和输出 shape 是什么?

标准回答：

设 TP degree 为 T，并假设 A、H 可整除。沿 QKV 权重输出维切分：

W_qkv_rank [H,3H/T]

QKV_rank [B,S,3H/T]

Q/K/V_rank [B,A/T,S,D]

每卡直接得到自己负责的完整 heads，因此 forward 不需要先合并 QKV 输出。

可能追问： Column-parallel linear 的 backward 在哪里通信?

易错点：不要看到输出分片就机械加入 All-Gather。

4.2.2 QH-A04 | 为什么 TP Attention core 可以本地计算?

Q： QK、softmax 和 PV 是否需要通信?

标准回答：

Head-parallel TP 给每卡分配 A/T 个完整 heads；每个 head 的 D 维和完整 sequence 都在本卡，因此：

Score_rank [B,A/T,S,S]

Prob_rank [B,A/T,S,S]

Context_rank [B,A/T,S,D] → [B,S,H/T]

这些操作在标准 MHA 假设下可本地完成。

可能追问：如果拆分的是 head dimension 而不是 heads，会发生什么?

易错点： “Attention 无通信” 只针对这种切分和 Attention core，不代表整个子层无通信。

4.2.3 QH-A05 | O projection 为什么是 row parallel?

Q：为什么最终要 All-Reduce，而不是 concat?

标准回答：

Context 已按 hidden 输入维分片为 [B,S,H/T]，因此每卡使用：

W_o_rank [H/T,H]

计算：

Y_partial_rank [B,S,H]

这里每卡只计算矩阵乘法归约维的一部分贡献，形状相同但数值不完整，所以需要 sum All-Reduce。Concat 适用于输出维分片，不适用于 partial sum。

可能追问： Bias 应在归约前还是归约后加？

易错点： Row/column 的名称要结合约定的权重 shape 解释。

4.2.4 QH-A06 | 普通 TP Attention 的 forward/backward 在哪里通信？

Q：请标出最关键的 collective。

标准回答：

Forward 中 QKV column-parallel 无通信，本地 Attention 无通信，O projection 的 partial output 需要 All-Reduce。Backward 中 row-parallel 层可从完整 dY 得到本地 dContext；QKV column-parallel 的每卡 dX 只是 partial，需要跨 TP ranks 求和。

可能追问：一个完整 Transformer layer 为什么还有 MLP 的通信？

易错点：回答时必须区分 forward、backward 和整层总通信次数。

4.2.5 QH-A07 | 给定数值时如何快速推导 TP shape？

Q： B=8、S=4096、H=8192、A=64、D=128、T=8 时，每卡 shape 是什么？

标准回答：

- 每卡 8 heads。
- W_qkv_rank：[8192,3072]。
- QKV_rank：[8,4096,3072]。
- 单个 Q/K/V：[8,8,4096,128]。
- Score：[8,8,4096,4096]。
- Context：[8,4096,1024]。
- W_o_rank：[1024,8192]。
- Y_partial：[8,4096,8192]。

可能追问：每卡计算量相对单卡如何变化？

易错点：先算 A/T 和 H/T，再逐算子推导，避免凭记忆写答案。

4.2.6 QH-A08 ｜如何计算 TP Attention 的显存与通信量?

Q：上述数值例子中 Score 和输出 All-Reduce 多大?

标准回答：

Score 每卡元素数为 8×8×4096×4096=1,073,741,824，bf16 约 2 GiB。输出 All-Reduce tensor 元素数为 8×4096×8192=268,435,456，bf16 约 512 MiB。8 卡 Ring All-Reduce 按每卡发送量口径约为 2×7/8×512=896 MiB。

可能追问：为什么通信量公式必须说明口径?

易错点： MiB 与 MB、发送量与发送加接收总量不能混用。

4.3 Sequence 与 Context Parallel

4.3.1 QH-A09 ｜经典 Megatron SP 如何改变数据布局?

Q：在 TP 基础上加入 Megatron sequence parallel 后发生什么?

标准回答：

层间 activation 沿 sequence 切分：

X_sp_rank [B,S/T,H]

进入 QKV column-parallel 前沿 sequence 做 All-Gather，恢复 [B,S,H]；本地执行 head-parallel Attention；O projection 得到 [B,S,H] partial 后，用沿 sequence 的 Reduce-Scatter 直接得到 [B,S/T,H]。

可能追问： LayerNorm 和 residual 在什么布局上执行?

易错点：经典 Megatron SP 通常复用 TP group，不一定是独立 world-size 维度。

4.3.2 QH-A10 ｜经典 Megatron SP 为什么没有消除 S^2?

Q： SP 后每卡的 Score 变成 [S/T,S/T] 吗?

标准回答：

不会。进入 Attention 前 sequence-sharded activation 已经 All-Gather 为完整 S，而 heads 仍按 TP 切分，所以每卡 Score 仍是 [B,A/T,S,S]。Megatron SP 主要减少 LayerNorm、dropout、residual 等非 TP 区域的 activation 冗余，不直接分解单个 head 的完整上下文计算。

可能追问：真正切分上下文通常使用什么方案?

易错点：不要把所有名为 sequence parallel 的实现视为同一算法。

4.3.3 QH-A11 | SP 的 forward 与 backward collective 如何对应?

Q: All-Gather 和 Reduce-Scatter 的反向是什么?

标准回答:

在相应线性映射中，forward All-Gather 的 backward 是 Reduce-Scatter；forward Reduce-Scatter 的 backward 是 All-Gather。这反映了复制/拼接布局与梯度归约/分片之间的伴随关系。工程实现可能异步或融合 collective，但逻辑对应关系不变。

可能追问: AG+RS 与一个 Ring AR 的带宽量级如何比较?

易错点: 带宽量级相近不代表 latency、buffer 和重叠效果完全相同。

4.3.4 QH-A12 | Ulysses Sequence Parallel 的核心是什么?

Q: Ulysses 如何在 sequence shard 和 head shard 间转换?

标准回答:

设独立 SP degree 为 P，开始时每卡拥有 [B,S/P,A,D]。通过 All-to-All 将布局转换为 [B,S,A/P,D]，每卡在完整 sequence 上计算部分 heads；完成后再用反向 All-to-All 回到 sequence-sharded 布局。它要求 head 数与并行度有合适关系。

可能追问: Ulysses 与 TP 同时使用时 process group 如何组织?

易错点: All-to-All 不是简单 All-Gather，它同时改变数据归属。

4.3.5 QH-A13 | Ring Attention 如何支持长上下文?

Q: 每卡不持有完整 K/V 时如何得到精确 Attention?

标准回答:

每卡保留本地 Q 和一段 K/V，K/V blocks 在设备间环形传递。每一步计算本地 Q 对当前 K/V block 的局部 score，并用 online softmax 更新行最大值、归一化和与累积输出。完成所有环步后得到对全局上下文的精确结果。

可能追问: 如何隐藏 K/V 传输?

易错点: Causal mask、负载均衡和通信计算重叠是实现难点。

4.3.6 QH-A14 | GQA 和 MQA 在 TP 下如何处理?

Q: KV heads 少于 Query heads 时还能直接按 T 切吗?

标准回答:

GQA 的 KV heads 少于 Query heads，MQA 通常只有一个 KV head。若 A_kv 可被 T 整除，可以按 KV heads 切；若 A_kv<T 或不能整除，可复制 K/V、限制 Attention 使用的 TP degree、使用独立映射或与其他并行维组合。不能机械写成 A_kv/T。

可能追问：复制 K/V 的显存和通信代价是什么?

易错点： Query head 数可整除不代表 KV head 数也可整除。

4.3.7 QH-A15 | Decode 阶段的 tensor shape 和瓶颈是什么?

Q：单 token decode 时 Q、KV cache 和 Score 通常是什么形状?

标准回答：

在 KV heads 可切的前提下，本地 Q 常为 [B,A/T,1,D]，KV cache 为 [B,A_kv/T,S_cache,D]，Score 为 [B,A/T,1,S_cache]。单步矩阵较小、算术强度较低，常更受 KV cache HBM 带宽、batch 调度和通信 latency 影响。

可能追问： Prefill 和 Decode 的优化重点为何不同?

易错点： Decode 的 sequence 长度 1 指当前 Query 长度，不代表 cache 长度为 1。

4.3.8 QH-A16 | 如何比较三类 SP/CP 方案?

Q： Megatron SP、Ulysses 和 Ring/Context Parallel 的主要区别是什么?

标准回答：

Megatron SP 在 TP 边界用 AG/RS，主要节省层间 activation；Ulysses 用 A2A 在 sequence shard 与 head shard 间转换；Ring/Context Parallel 保持 sequence 分片，通过传递 K/V blocks 和 online softmax 覆盖全局上下文。选择取决于 head 数、上下文长度、拓扑、显存和通信重叠能力。

可能追问：哪个方案最依赖 A2A 性能？哪个最依赖 P2P pipeline?

易错点：面试时先确认对方所说 SP 的具体定义。

第五章　通信与分布式高性能计算

5.1　Collective 与性能模型

5.1.1　QH-C01 ｜四种核心 collective 分别做什么?

Q：请解释 All-Reduce、All-Gather、Reduce-Scatter 和 All-to-All。

标准回答：

All-Reduce 让每卡得到完整归约结果；All-Gather 把各卡分片拼成完整 tensor 并复制到每卡；Reduce-Scatter 先归约，再让每卡保留一个结果分片；All-to-All 让每卡向不同 peer 发送不同分片并接收对应分片。它们分别服务于求和、布局聚合、归约分片和多对多重排。

可能追问： TP、SP、DP、MoE 各使用哪些 collective?

易错点： Collective 名称描述数据语义，不等于固定的一种底层算法。

5.1.2　QH-C02 ｜为什么 All-Reduce 等于 RS+AG?

Q：从结果布局解释这个等价关系。

标准回答：

Reduce-Scatter 先完成逐元素归约，并把最终结果按块分给不同 ranks；此时每卡只有完整归约结果的一部分。All-Gather 再交换这些结果分片，使每卡获得完整归约 tensor。因此语义上 AR = RS + AG，Ring All-Reduce 也常按这两个阶段实现。

可能追问：为什么 SP 可以直接保留 RS 后的分片而省掉 AG?

易错点：算法等价不代表每个框架都以完全相同的 pipeline 实现。

5.1.3　QH-C03 ｜ α-β 通信模型是什么?

Q：小消息和大消息分别受什么限制?

标准回答：

常用简化模型为：

T_comm ≈ α× 轮次或消息开销 + β× 传输字节数

α 表示固定 latency、软件和同步开销；β 是每字节传输时间，约为有效带宽的倒数。小消息中 α 占比更高，大消息更接近带宽限制。

可能追问：为什么把一个 tensor 切成过多小 bucket 会变慢?

易错点： α、β 是有效模型参数，不等于硬件手册上的单一数字。

5.1.4 QH-C04 ｜ Ring collective 的通信量是多少?

Q： tensor 大小为 N bytes、p 个 ranks 时，每卡发送多少?

标准回答：

Ring Reduce-Scatter 与 All-Gather 每卡分别发送约 (p-1)/p×N bytes；Ring All-Reduce 由两阶段组成，每卡发送约 2(p-1)/p×N bytes。公式中的 N 通常指每卡 collective 输入 tensor 的逻辑大小。

可能追问：接收量、链路总流量如何计算?

易错点：必须说明发送、接收或二者之和的统计口径。

5.1.5 QH-C05 ｜ Ring 和 Tree 各适合什么场景?

Q：为什么 collective 库需要根据消息大小选择算法?

标准回答：

Ring 在大消息下可让各链路持续传输，带宽利用通常较好，但步骤数为 O(p)，小消息时 latency 更明显。Tree 类算法步骤约 O(log p)，可降低小消息的轮次开销，但带宽利用与拓扑适配可能不同。真实选择还取决于节点层级、channel 数和协议。

可能追问：分层 collective 如何利用节点内外不同链路?

易错点：不要断言 Ring 永远比 Tree 快或反之。

5.2 拓扑、重叠与扩展

5.2.1 QH-C06 ｜为什么 TP 通常放在节点内?

Q： TP、DP process group 应如何考虑拓扑?

标准回答：

TP 通信通常每层发生、频率高且处于关键路径，对 latency 和带宽敏感，因此优先映射到 NVLink/NVSwitch 等节点内高速链路。DP 梯度同步频率相对低且更容易分 bucket 重叠，通常更能容忍跨节点网络。最终仍要结合模型、NIC 数量、rank mapping 和拓扑测量。

可能追问：只有 PCIe 的机器如何调整策略?

易错点：只说 “有 NVLink” 不够，要知道哪些 GPU 直连及跨节点路径。

5.2.2 QH-C07 | 异步发起等于通信被隐藏了吗?

Q：通信计算重叠需要哪些条件?

标准回答：

异步 API 只说明调用者无需立即等待。真正重叠需要通信数据已 ready，后续存在不依赖通信结果的计算，使用合适 stream 和 event 依赖，并且通信没有与计算严重争用 SM、HBM 或链路。最终必须用时间线确认关键路径上暴露的等待时间。

可能追问： Bucket 太大、太小分别有什么问题?

易错点：代码中出现 async_op 不能作为重叠成功的证据。

5.2.3 QH-C08 | DP、TP、PP、SP、CP、EP 分别切什么?

Q：请从切分对象和主要通信比较并行策略。

标准回答：

DP 切 batch，主要同步梯度；TP 切单层权重和 hidden/head，每层使用 AR/AG/RS；PP 切 layers，stage 间 send/recv；SP/CP 切 sequence/context，使用 AG/RS、A2A 或 P2P；EP 切 experts，通常通过 A2A dispatch tokens。组合策略要让高频通信匹配快链路。

可能追问：经典 Megatron SP 为什么不一定增加 world size?

易错点：名称相同的 SP 在不同框架中可能指不同布局。

5.2.4 QH-C09 | Strong scaling 与 weak scaling 是什么?

Q：如何评价增加 GPU 后的扩展效率?

标准回答：

Strong scaling 固定总问题规模，理想时间随 p 下降，效率为：

$$\text{Efficiency_strong} = T_1 / (p \times T_p)$$

Weak scaling 固定每卡问题规模，理想 step time 不变，常用 T_1/T_p 评价。扩展变差可能来自通信占比上升、每卡 GEMM 变小、跨节点链路、负载不均、pipeline bubble 或 CPU 输入瓶颈。

可能追问：为什么 strong scaling 通常更难维持?

易错点：比较效率时必须保持问题规模定义一致。

5.2.5 QH-C10 | MFU 与 HFU 有什么区别?

Q：如何解释训练系统的计算利用率?

标准回答：

MFU 通常等于实际每秒完成的模型理论 FLOPs 除以对应精度的硬件峰值，强调有效模型工作；HFU 可把重计算等实际执行的额外 FLOPs 计入，所以 activation recomputation 场景中可能高于 MFU。报告时应说明 FLOPs 公式、精度峰值、padding、重计算和通信是否计入。

可能追问： MFU 高是否代表端到端训练成本一定低?

易错点：不同论文或框架的 FLOPs 口径可能不同。

第六章　项目深挖与 Vibe Coding

6.1　项目表达

6.1.1　QH-P01｜如何用 60 秒介绍性能项目？

Q：项目介绍应该按什么结构组织？

标准回答：

使用 “背景与规模 → 个人职责 → baseline 与瓶颈证据 → 核心方案 → 量化结果 → 难点” 结构。必须包含硬件、典型 shape、至少两个性能数字，并明确哪些部分是自己完成的。结论先行，避免从项目历史开始流水账。

可能追问：如何扩展成 5 分钟版本？

易错点：百分比提升必须同时给 baseline 和测试条件。

6.1.2　QH-P02｜如何证明你找到了真正瓶颈？

Q： “GPU 利用率低” 为什么不是充分答案？

标准回答：

应建立证据链：端到端时间线定位热点与空洞；对热点 kernel 查看 FLOPs、带宽、stall、occupancy；对通信检查 tensor 大小、collective、拓扑和重叠；结合 Roofline；最后通过改变 shape、dtype、layout、GPU 数或关闭某项优化做控制变量实验。

可能追问：如果 profiler 指标相互矛盾怎么办？

易错点： GPU utilization、SM utilization、Tensor Core utilization 是不同概念。

6.1.3　QH-P03｜如何证明优化结果可信？

Q：项目性能和正确性需要提供哪些信息？

标准回答：

性能要给相同语义、shape、dtype、硬件下的 baseline，说明 warmup、同步、重复次数和统计值；报告 latency、throughput、显存及必要的 MFU。正确性要给参考实现、误差阈值、边界 shape、不同 dtype、并行一致性和端到端回归。

可能追问：为什么只测单一 shape 不可信?

易错点：速度提升不能来自减少功能、降低精度或错误计时。

6.1.4 QH-P04 ｜失败案例和个人贡献怎么讲?

Q：面试官为什么喜欢追问失败方案?

标准回答：

失败案例能体现测量、推理和纠错能力。按 “原假设 → 实验 → 反例证据 → 根因 → 修正 → 经验” 回答。个人贡献使用 “我定位、我设计、我实现、我验证”，同时坦诚团队接口和非本人负责范围。

可能追问：如果重做一次会先改变什么?

易错点：不要选无关紧要、没有技术判断的 “伪失败”。

6.2 Vibe Coding

6.2.1 QH-V01 ｜如何评价 Vibe Coding?

Q： AI 编程在工程中的定位是什么?

标准回答：

它是生成、搜索和探索的加速器，不是正确性保证者。适合样板代码、测试、benchmark、日志分析、文档和候选实现；并行语义、数值关键路径、安全操作和上线决策必须由工程师审查、执行验证并承担责任。

可能追问：底层性能工程为什么仍需要人工判断?

易错点：避免 “全部交给 AI” 或 “完全不用 AI” 两种极端表达。

6.2.2 QH-V02 ｜可靠的 AI 编程工作流是什么?

Q：怎样减少模型误解上下文和大范围误改?

标准回答：

先明确接口、shape、非目标和验收标准；让模型先读代码并复述约束；拆成小改动，尽量测试先行；逐步审查 diff，真实运行编译、测试和 benchmark；最后汇报证据、假设和未覆盖风险。使用小 commit、分支或 worktree 保证可回退。

可能追问：怎样提示模型不要马上修改?

易错点：模型声称 “测试通过” 不等于存在真实执行证据。

6.2.3　QH-V03 ｜如何验证 AI 生成的 GPU kernel?

Q：代码能运行且快就可以接受吗?

标准回答：

需要检查 pointer arithmetic、stride、mask、越界、同步、累加精度和数值稳定性；用典型与非整齐 shape、不同 dtype、连续和非连续输入验证；使用公平 baseline、正确异步计时和 profiler；还要检查寄存器 spilling、资源占用与多 shape 性能。

可能追问： AI 报告提升 30% 时先核对什么?

易错点：单次 allclose 和单一 shape benchmark 都不足以验收。

6.2.4　QH-V04 ｜使用 AI 时有哪些安全边界?

Q：公司项目中如何保护敏感信息?

标准回答：

不向未经授权的服务提交密钥、生产数据、用户信息或公司私有代码；遵守组织的数据和模型使用政策；对删除、迁移、权限、部署等高风险操作要求明确授权和可恢复方案。即使代码由 AI 生成，安全、许可和上线责任仍属于工程团队。

可能追问：如何处理第三方生成代码的许可证风险?

易错点：开发效率不能覆盖数据合规和供应链风险。

腾讯 HPC 面试刷题题库

项目深挖・GPU ・Triton ・TP/SP ・分布式通信

面试备战资料

2026-10-21

目录

第一章 使用说明与自测方法 **4**
1.1 题目分级 4
1.2 自测评分 4

第二章 项目深挖题 **5**
2.1 项目概述 5
2.2 瓶颈与优化 5
2.3 结果与可信度 5
2.4 失败与复盘 6

第三章 GPU 架构与性能题 **7**
3.1 执行模型 7
3.2 存储体系 7
3.3 计算与性能模型 7
3.4 优化与分析 8

第四章 Triton 题 **9**
4.1 编程模型 9
4.2 Softmax 与 Reduction 9
4.3 Matmul 与调优 9
4.4 编码与实验 9

第五章 Attention、TP 与 SP 题 **11**
5.1 单卡与 TP 推导 11
5.2 数值白板题 11
5.3 Sequence/Context Parallel 12
5.4 GQA、MQA 与推理 12

第六章 通信与分布式计算题 **13**
6.1 Collective 与公式 13
6.2 拓扑与重叠 13
6.3 并行与扩展 13
6.4 通信计算题 14

第七章 Vibe Coding 题 **15**

第八章 模拟面试套题 **16**

8.1 套题一：基础闭环 16
8.2 套题二：性能深挖 16
8.3 套题三：压力追问 16

本精简版未收录原题库第九、十章（答案要点与评分锚点、最终达标清单）。

第一章　使用说明与自测方法

这份题库面向 GPU 高性能计算、大模型并行与 Triton 岗位。建议先闭卷作答，再到后半部分核对答案要点。

1.1　题目分级

标记	含义	完成标准
P0	本轮面试已经直接涉及	能在白板上完整作答
P1	高频扩展问题	能解释原理、代价和边界
P2	加分题	能给出合理分析路径

1.2　自测评分

每题按 0～4 分评分：

分数	标准
0	完全不会
1	只记得名词
2	能讲概念，缺少公式、shape 或证据
3	能完整回答，并说明关键条件
4	能处理追问、边界、代价和替代方案

建议建立错题表：

题号	首次得分	错误原因	二刷得分	三刷得分

第二章　项目深挖题

这些题没有统一答案。作答时必须使用自己项目的真实数字、Profiler 证据和个人贡献。

2.1　项目概述

1. **P-01** ｜ **P0**：用 60 秒介绍你最重要的项目。
2. **P-02** ｜ **P0**：这个项目解决什么问题？用户或下游系统是谁？
3. **P-03** ｜ **P0**：你个人负责的边界是什么？哪些部分不是你做的？
4. **P-04** ｜ **P0**：典型输入 shape、模型规模、GPU 型号、卡数和互联方式是什么？
5. **P-05** ｜ **P0**：项目最核心的技术难点是什么？

2.2　瓶颈与优化

6. **P-06** ｜ **P0**：优化前的 baseline 是什么？
7. **P-07** ｜ **P0**：你如何判断瓶颈是计算、访存、launch、同步还是通信？
8. **P-08** ｜ **P0**：Nsight Systems 和 Nsight Compute 分别提供了什么证据？
9. **P-09** ｜ **P0**：核心优化为什么有效？减少了哪些 FLOPs、HBM bytes、launch 或等待？
10. **P-10** ｜ **P1**：为什么没有选择另一个方案？
11. **P-11** ｜ **P1**：这个优化在哪些 shape 上无效或变慢？
12. **P-12** ｜ **P1**：如果 GPU 型号更换，结论是否仍成立？

2.3　结果与可信度

13. **P-13** ｜ **P0**：优化前后 latency、throughput、显存和 MFU 分别是多少？
14. **P-14** ｜ **P0**：benchmark 如何 warmup、同步、重复和统计？
15. **P-15** ｜ **P0**：如何验证数值正确性和端到端正确性？
16. **P-16** ｜ **P1**：精度误差阈值为什么合理？
17. **P-17** ｜ **P1**：扩到更多 GPU 后，最先出现的瓶颈是什么？

2.4　失败与复盘

18. **P-18** ｜ **P0**：讲一个失败的优化或错误判断。
19. **P-19** ｜ **P1**：如果重新做一次，你会先测量什么？
20. **P-20** ｜ **P1**：还有哪些优化空间？为什么目前没有继续做？

第三章　GPU 架构与性能题

3.1　执行模型

1. **G-01** ｜ **P0**：画出 GPU、GPC、SM、warp scheduler、CUDA Core、Tensor Core 的层级关系。
2. **G-02** ｜ **P0**：grid、block、warp、thread 分别是什么？block 如何调度到 SM？
3. **G-03** ｜ **P0**：SIMT 与 SIMD 有什么区别？
4. **G-04** ｜ **P0**：什么是 warp divergence？所有分支都会导致明显损失吗？
5. **G-05** ｜ **P1**：GPU 如何用大量 warp 隐藏指令和访存延迟？

3.2　存储体系

6. **G-06** ｜ **P0**：画出 register、shared memory、L1、L2、HBM 的层级、作用域和生命周期。
7. **G-07** ｜ **P0**：什么是 global memory coalescing？举一个好例子和坏例子。
8. **G-08** ｜ **P0**：什么是 shared-memory bank conflict？如何避免？
9. **G-09** ｜ **P0**：register pressure 为什么会影响 occupancy？
10. **G-10** ｜ **P0**：occupancy 越高性能一定越好吗？
11. **G-11** ｜ **P1**：shared memory 一定比 L1 快吗？
12. **G-12** ｜ **P1**：为什么矩阵转置常使用带 padding 的 shared-memory tile？

3.3　计算与性能模型

13. **G-13** ｜ **P0**：CUDA Core 与 Tensor Core 有什么区别？
14. **G-14** ｜ **P0**：什么是 arithmetic intensity？
15. **G-15** ｜ **P0**：写出 Roofline Model，并说明如何判断 compute-bound 和 memory-bound。
16. **G-16** ｜ **P1**：实际性能远低于两条 Roofline 上限时，可能是什么原因？
17. **G-17** ｜ **P0**：为什么 GEMM 使用 tiling 后 arithmetic intensity 会提高？
18. **G-18** ｜ **P0**：为什么小矩阵通常跑不满 GPU？
19. **G-19** ｜ **P1**：fp16、bf16、fp8 在吞吐、动态范围和数值稳定性上如何权衡？
20. **G-20** ｜ **P1**：Ampere 与 Hopper 在大模型计算相关能力上有哪些重要差异？

3.4　优化与分析

21. **G-21** ｜ **P0**：kernel fusion 为什么可能加速?
22. **G-22** ｜ **P1**：为什么融合过多反而可能变慢?
23. **G-23** ｜ **P0**：FlashAttention 为什么省显存、为什么更快?
24. **G-24** ｜ **P0**：FlashAttention 是否减少了 Attention 主项 FLOPs?
25. **G-25** ｜ **P1**：说明 online softmax 需要维护哪些状态。
26. **G-26** ｜ **P0**：Nsight Systems 和 Nsight Compute 的职责有什么不同?
27. **G-27** ｜ **P0**：看到 GPU 时间线有空洞，应按什么顺序排查?
28. **G-28** ｜ **P1**：GPU kernel 是异步执行的，benchmark 应如何计时?
29. **G-29** ｜ **P1**：如何证明一个 kernel 是 memory-bound，而不是凭感觉判断?
30. **G-30** ｜ **P2**：一个 kernel occupancy 下降但速度提高，给出三种合理解释。

第四章　Triton 题

4.1　编程模型

1. **T-01** ｜ **P0**：Triton program instance 与 CUDA block 有什么关系？
2. **T-02** ｜ **P0**：tl.program_id、tl.arange、tl.load、tl.store 分别做什么？
3. **T-03** ｜ **P0**：为什么非整齐 shape 需要 mask？
4. **T-04** ｜ **P1**：mask 是如何参与 load、store 和 reduction 的？
5. **T-05** ｜ **P1**：Triton 相比 CUDA 抽象掉了什么，又没有抽象掉什么？

4.2　Softmax 与 Reduction

6. **T-06** ｜ **P0**：白板写出 row-wise fused softmax。
7. **T-07** ｜ **P0**：softmax 为什么要先减去行最大值？
8. **T-08** ｜ **P0**：padding lane 为什么通常填负无穷？
9. **T-09** ｜ **P1**：为什么 BLOCK_SIZE 常取不小于 n_cols 的 2 的幂？
10. **T-10** ｜ **P1**：一行太长，无法由单个 program 高效处理时怎么办？
11. **T-11** ｜ **P1**：reduction 为什么容易增加 register pressure？

4.3　Matmul 与调优

12. **T-12** ｜ **P0**：给定 C[M,N]=A[M,K]×B[K,N]，解释 BLOCK_M/N/K 的含义。
13. **T-13** ｜ **P0**：num_warps 和 num_stages 分别控制什么？
14. **T-14** ｜ **P1**：为什么 tile 越大不一定越快？
15. **T-15** ｜ **P1**：grouped ordering 为什么可能改善 cache reuse？
16. **T-16** ｜ **P0**：如何设计 autotune 的候选配置和 key？
17. **T-17** ｜ **P0**：一个 Triton kernel 比 PyTorch baseline 慢，如何系统排查？
18. **T-18** ｜ **P1**：如何识别 register spilling？

4.4　编码与实验

19. **T-19** ｜ **P0**：手写 vector add，并正确处理尾块。

20. **T-20** | **P0**：手写 fused softmax，并验证数值稳定性。
21. **T-21** | **P1**：写出 tiled matmul 的核心循环和 accumulator。
22. **T-22** | **P1**：设计 RMSNorm forward，说明累加精度和边界处理。
23. **T-23** | **P1**：为 vector add 推导最小 HBM bytes 和有效带宽。
24. **T-24** | **P1**：设计覆盖连续、非连续、非整齐 shape 和不同 dtype 的测试矩阵。
25. **T-25** | **P2**：一个融合 kernel 在 H=4096 快、H=8192 慢，设计排查实验。

第五章　Attention、TP 与 SP 题

统一符号：

> B 为 batch，S 为 sequence length，H 为 hidden size，A 为 Query head 数，D=H/A，T 为 TP degree。

除非题目另有说明，假设标准 MHA、权重记为 [in_dim,out_dim]、输入为 [B,S,H]、head 数可被 T 整除。

5.1　单卡与 TP 推导

1. **A-01** ｜ **P0**：写出单卡 MHA 从 X 到 Y 的全部 tensor shape。
2. **A-02** ｜ **P0**：QKV projection 的 FLOPs 量级是什么？
3. **A-03** ｜ **P0**：Attention 核心 QK^T 和 Prob×V 的 FLOPs 量级是什么？
4. **A-04** ｜ **P0**：TP 下为什么 QKV 通常使用 column parallel？
5. **A-05** ｜ **P0**：每卡 W_qkv、QKV、Q/K/V 的 shape 是什么？
6. **A-06** ｜ **P0**：为什么按完整 heads 切分后 Attention 核心不需要通信？
7. **A-07** ｜ **P0**：每卡 Score、Prob、Context 的 shape 是什么？
8. **A-08** ｜ **P0**：O projection 为什么使用 row parallel？
9. **A-09** ｜ **P0**：为什么 O projection 后需要求和，而不是 concat？
10. **A-10** ｜ **P0**：普通 TP Attention forward 的关键 collective 在哪里？
11. **A-11** ｜ **P0**：backward 的关键 collective 在哪里？

5.2　数值白板题

使用：

> B=8，S=4096，H=8192，A=64，D=128，T=8，dtype=bf16。

12. **A-12** ｜ **P0**：每卡负责多少个 heads？
13. **A-13** ｜ **P0**：写出每卡 W_qkv 和 QKV 的 shape。
14. **A-14** ｜ **P0**：写出每卡单个 Q/K/V 与 Score 的 shape。
15. **A-15** ｜ **P0**：写出 Context、W_o 和 Y_partial 的 shape。
16. **A-16** ｜ **P0**：若显式保存 Score，每卡需要多少元素和多少 GiB？
17. **A-17** ｜ **P0**：输出 All-Reduce 的逻辑 tensor 有多少元素、多少 MiB？
18. **A-18** ｜ **P1**：8 卡 Ring All-Reduce 按每卡发送量口径需要多少 MiB？

5.3 Sequence/Context Parallel

19. **A-19** | **P0**：经典 Megatron SP 的层间 activation shape 是什么?
20. **A-20** | **P0**：进入 QKV 前做什么 collective，shape 如何变化?
21. **A-21** | **P0**：O projection 后做什么 collective，shape 如何变化?
22. **A-22** | **P0**：经典 Megatron SP 为什么没有直接消除 S^2 Attention activation?
23. **A-23** | **P1**：SP forward 中 All-Gather 和 Reduce-Scatter 在 backward 分别对应什么?
24. **A-24** | **P1**：Ulysses 如何通过 All-to-All 在 sequence shard 与 head shard 之间转换?
25. **A-25** | **P1**：Ring Attention 如何在不聚合完整 K/V 的情况下得到精确结果?

5.4 GQA、MQA 与推理

26. **A-26** | **P0**：GQA 与 MQA 分别是什么?
27. **A-27** | **P1**：KV head 数小于或不能整除 TP degree 时怎么办?
28. **A-28** | **P1**：Decode 阶段本地 Q、KV cache 和 Score 的典型 shape 是什么?
29. **A-29** | **P1**：为什么 Decode 更容易受到 KV cache 带宽和通信 latency 影响?
30. **A-30** | **P2**：比较 Megatron SP、Ulysses 和 Ring/Context Parallel 的通信与显存特征。

第六章　通信与分布式计算题

6.1　Collective 与公式

1. **C-01** ｜ **P0**：解释 All-Reduce、All-Gather、Reduce-Scatter、All-to-All。
2. **C-02** ｜ **P0**：为什么 All-Reduce 可以拆成 Reduce-Scatter + All-Gather?
3. **C-03** ｜ **P0**：写出 α-β 通信模型，并解释两项含义。
4. **C-04** ｜ **P0**：设 tensor 为 N bytes、p 个 rank，写出 Ring AG、RS、AR 的每卡发送量。
5. **C-05** ｜ **P1**：Ring 和 Tree collective 分别更适合什么场景?
6. **C-06** ｜ **P1**：为什么 All-to-All 往往比 Ring All-Reduce 更难优化?

6.2　拓扑与重叠

7. **C-07** ｜ **P0**：NVLink、NVSwitch、PCIe、InfiniBand/RoCE 分别处于什么位置?
8. **C-08** ｜ **P0**：为什么 TP group 通常优先放在节点内?
9. **C-09** ｜ **P1**：rank mapping 为什么会影响性能?
10. **C-10** ｜ **P0**：异步发起通信是否等于成功隐藏通信?
11. **C-11** ｜ **P0**：通信与计算重叠成立需要哪些条件?
12. **C-12** ｜ **P1**：gradient bucket 太大和太小分别有什么问题?
13. **C-13** ｜ **P1**：如何用 Nsight Systems 证明通信真正被隐藏?

6.3　并行与扩展

14. **C-14** ｜ **P0**：DP、TP、PP、SP、CP、EP 分别切分什么?
15. **C-15** ｜ **P1**：经典 Megatron SP 为什么通常不作为独立 world-size 乘数?
16. **C-16** ｜ **P0**：定义 strong scaling 和 weak scaling。
17. **C-17** ｜ **P0**：写出 strong-scaling efficiency。
18. **C-18** ｜ **P1**：GPU 增加后性能扩展变差，列出至少五个原因。
19. **C-19** ｜ **P0**：MFU 是什么？计算时要说明哪些口径?
20. **C-20** ｜ **P1**：MFU 与 HFU 有什么区别?

6.4　通信计算题

21. **C-21** ｜ **P0**：bf16 tensor [8,4096,8192] 有多少 bytes 和 MiB?
22. **C-22** ｜ **P0**：8 卡 Ring All-Reduce 的每卡发送量是多少?
23. **C-23** ｜ **P1**：若有效链路带宽为 200 GB/s，忽略 latency，估算 C-22 的理想时间下限。
24. **C-24** ｜ **P1**：加入每轮 10 μs latency 后，为什么小 tensor 更不利?
25. **C-25** ｜ **P2**：设计一个实验，区分通信带宽瓶颈、collective latency 瓶颈和计算无法隐藏通信。

第七章　Vibe Coding 题

1. **V-01** ｜ **P0**：你如何在实际项目中使用 AI 编程？
2. **V-02** ｜ **P0**：哪些任务适合交给 AI，哪些不适合？
3. **V-03** ｜ **P0**：如何防止 AI 生成不存在的 API 或误解项目约束？
4. **V-04** ｜ **P0**：如何验证 AI 生成的 GPU/Triton kernel？
5. **V-05** ｜ **P1**：AI 声称 benchmark 提升 30%，你会检查什么？
6. **V-06** ｜ **P1**：如何控制大规模修改的风险？
7. **V-07** ｜ **P1**：如何处理密钥、生产数据和公司代码？
8. **V-08** ｜ **P2**：如何评价 “Vibe Coding 会取代底层性能工程师”？

第八章　模拟面试套题

8.1　套题一：基础闭环

建议限时 45 分钟。

1. 60 秒项目介绍。
2. 用 Profiler 证据解释项目的核心瓶颈。
3. 画 GPU memory hierarchy。
4. 解释 coalescing、bank conflict 和 occupancy。
5. 推导 B=8、S=4096、H=8192、A=64、T=8 下的 TP Attention。
6. 加入 Megatron SP 后重新标出 shape 与 collective。
7. 写 Triton softmax 的核心逻辑。
8. 解释 All-Reduce 的 Ring 通信量。
9. 用 30 秒回答 Vibe Coding 的使用原则。

8.2　套题二：性能深挖

建议限时 60 分钟。

1. 讲一个性能优化失败案例。
2. 用 Roofline 分析一个 elementwise kernel 和一个 GEMM。
3. 解释 FlashAttention 的 online softmax。
4. 一个 kernel occupancy 从 75% 降到 50%，但快了 20%，如何分析？
5. Triton matmul 在小 shape 下比 cuBLAS 慢，为什么？
6. 解释 TP forward/backward 的通信对称关系。
7. 比较 Megatron SP、Ulysses 和 Ring Attention。
8. 多机扩展效率突然下降，给出排查树。
9. 设计一套可信的 benchmark。

8.3　套题三：压力追问

建议限时 60 分钟，要求每题继续自问 “为什么” 和 “代价是什么”。

1. 项目提升 30% 的数字可靠吗？逐项证明。

2. 为什么不是 CPU、数据加载或同步瓶颈?
3. 为什么 QKV column parallel 不需要 forward All-Gather?
4. 为什么 O projection 不能 concat?
5. GQA 的 KV heads 少于 TP degree 怎么办?
6. SP 是否一定降低通信量?
7. All-to-All 为什么可能被热点流量拖慢?
8. fusion 为什么可能降低 occupancy?
9. AI 生成的 kernel 正确且更快，是否可以直接上线?
10. 如果只能再准备一天，你会用什么证据证明自己适合这个岗位?